Data Structures and Algorithms I.



University of Szeged

Robert Nagy

2025/26. Spring Semester



Dynamic Programming

Fibonacci numbers

1, 1, 2, 3, 5, 8, 13, 21 , 34 ...

Fibonacci numbers

$1, 1, 2, 3, 5, 8, 13, 21 , 34 \ldots$

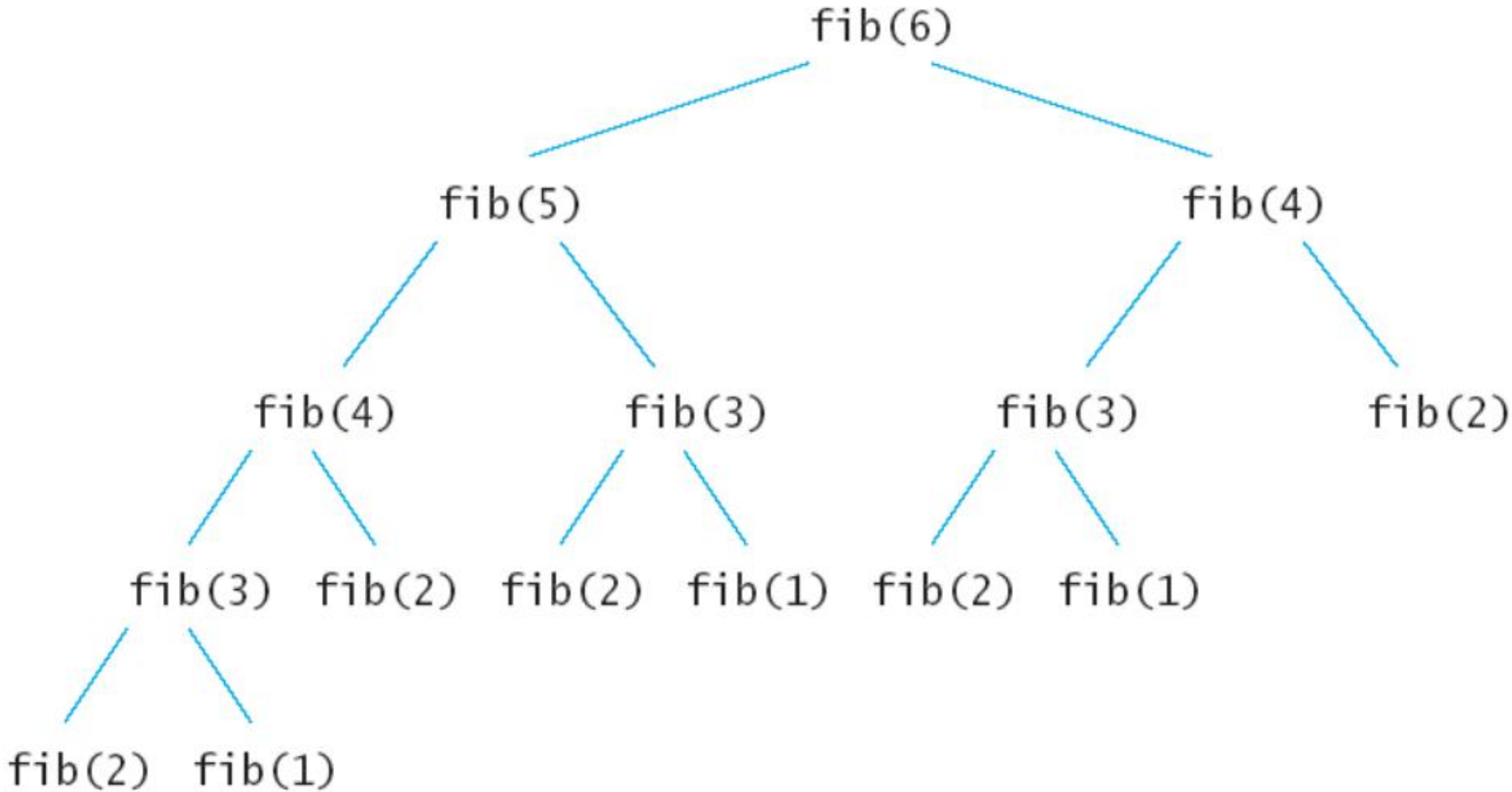
$fibo(1) = 1$
$fibo(2) = 1$

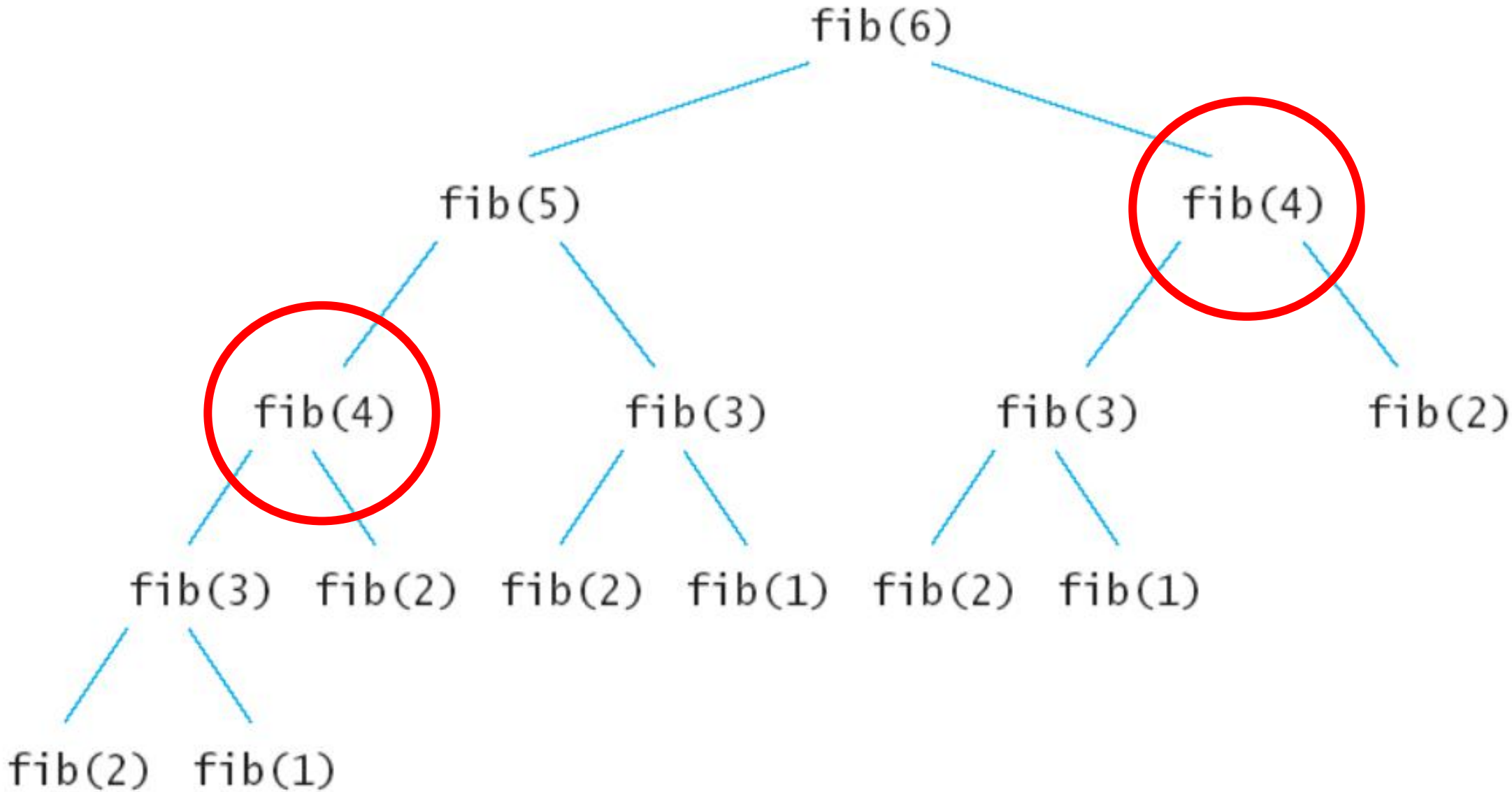
$fibo(n) = fibo(n-1) + fibo(n-2)$

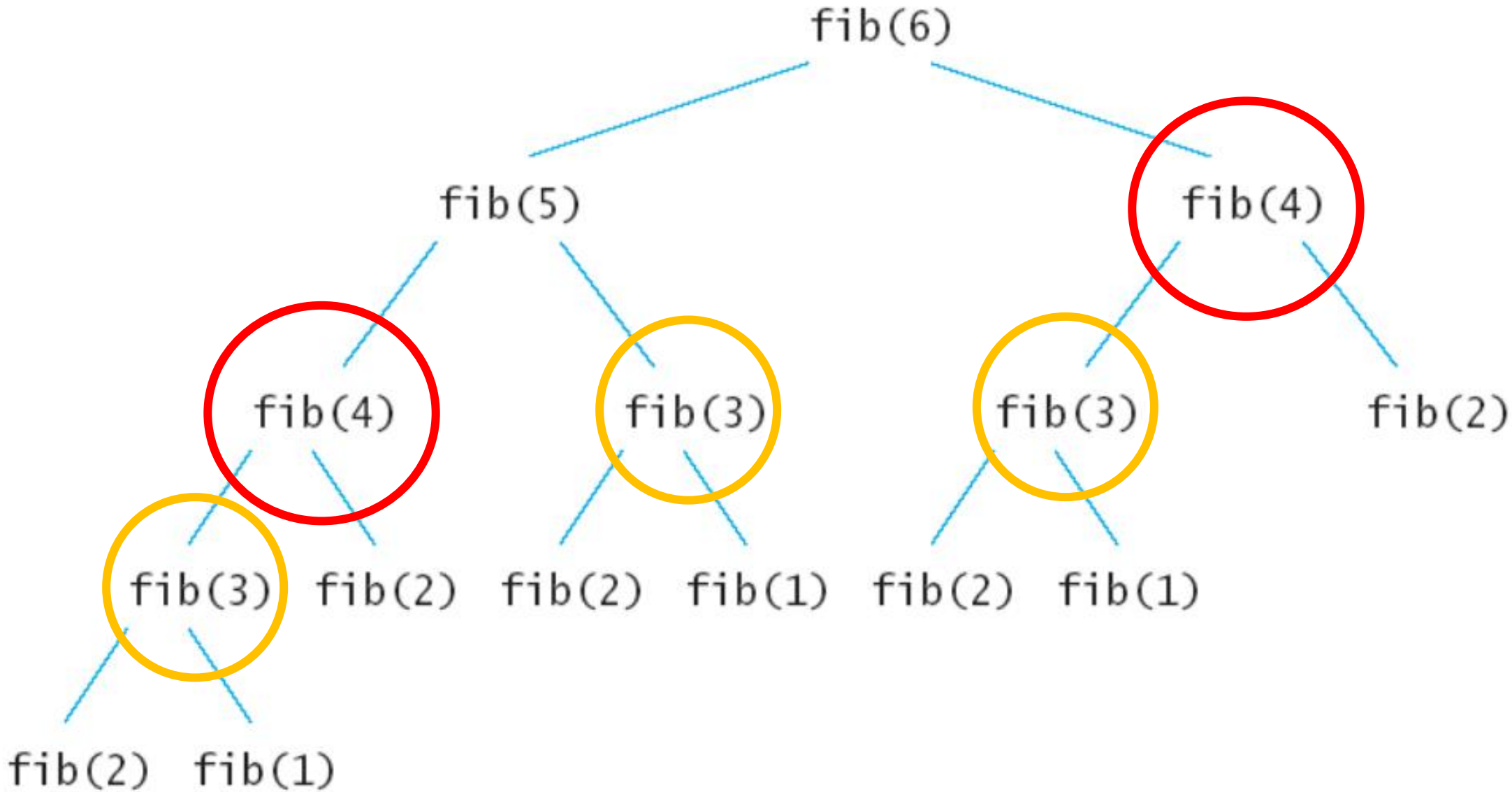
main.py

```
1 def fibo(n):
2     if n <= 2:
3         return 1
4     return fibo(n-1) + fibo(n-2)
5
```

https://www.artofcse.com/learning/introduction-to-dynamic-programming




fib(6)
fib(5)
fib(4)
fib(4)
fib(3)
fib(3)
fib(2)
fib(3)
fib(2)
fib(2)
fib(1)
fib(2)
fib(1)
fib(2)
fib(1)


fib(6)
fib(5)
fib(4)
fib(4)
fib(3)
fib(3)
fib(2)
fib(3)
fib(2)
fib(2)
fib(1)
fib(2)
fib(1)
fib(2)
fib(1)

D.R.Y. – Don’t Repeat Yourself

Memoization

The solution we need!

```
1   memo = [False] * 128
```

```
memo = [False] * 128
last_known_elem = 3
memo[1] = 1
memo[2] = 1
```

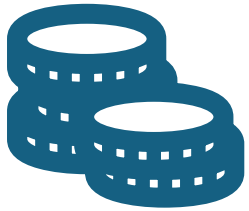
```
memo = [False] * 128
last_known_elem = 3
memo[1] = 1
memo[2] = 1
def fibo(n):
    global memo, last_known_elem
    if memo[n]:
        return memo[n]
```

```
memo = [False] * 128
last_known_elem = 3
memo[1] = 1
memo[2] = 1
def fibo(n):
    global memo, last_known_elem
    if memo[n]:
        return memo[n]
    for i in range(last_known_elem, n + 1):
        memo[i] = memo[i - 1] + memo[i - 2]
```

```
memo = [False] * 128
last_known_elem = 3
memo[1] = 1
memo[2] = 1
def fibo(n):
    global memo, last_known_elem
    if memo[n]:
        return memo[n]
    for i in range(last_known_elem, n + 1):
        memo[i] = memo[i - 1] + memo[i - 2]
    last_known_elem = n
    return memo[n]

```

Optimization problems



Coin Change problem

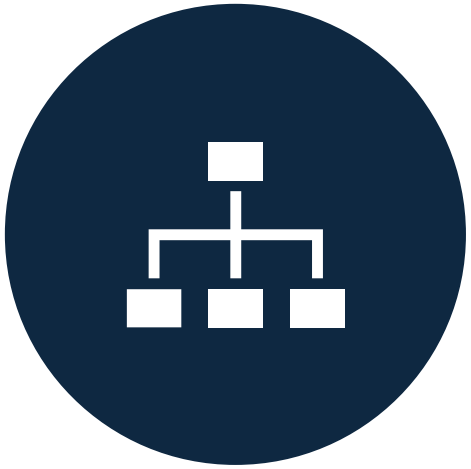
Example

https://www.youtube.com/watch?v=jgiZlGzXMBw

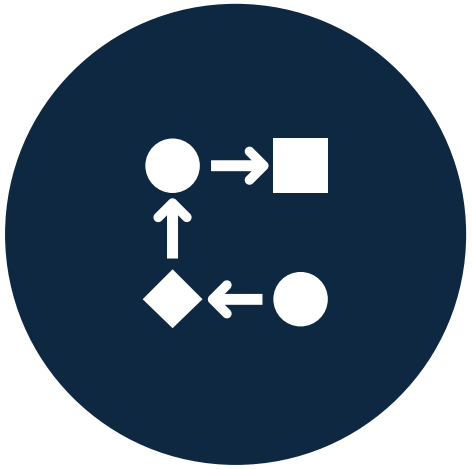
$$coins = [1, 2, 5]$$

$$target\ amount = 11$$

Two approaches



TOP-DOWN



BOTTOM-UP

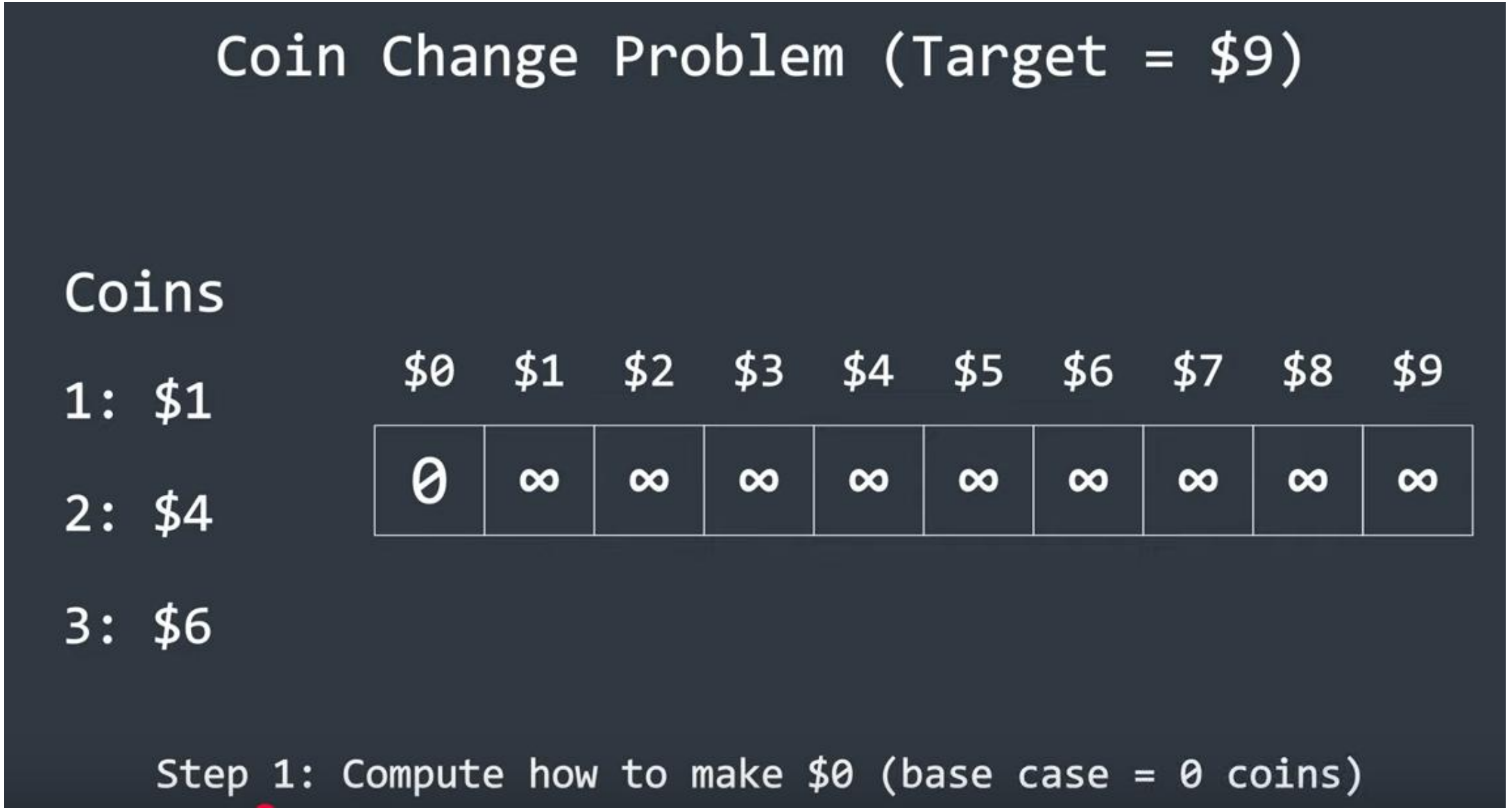

It’s drawing time!

https://www.youtube.com/watch?v=KnWorqyDSLA

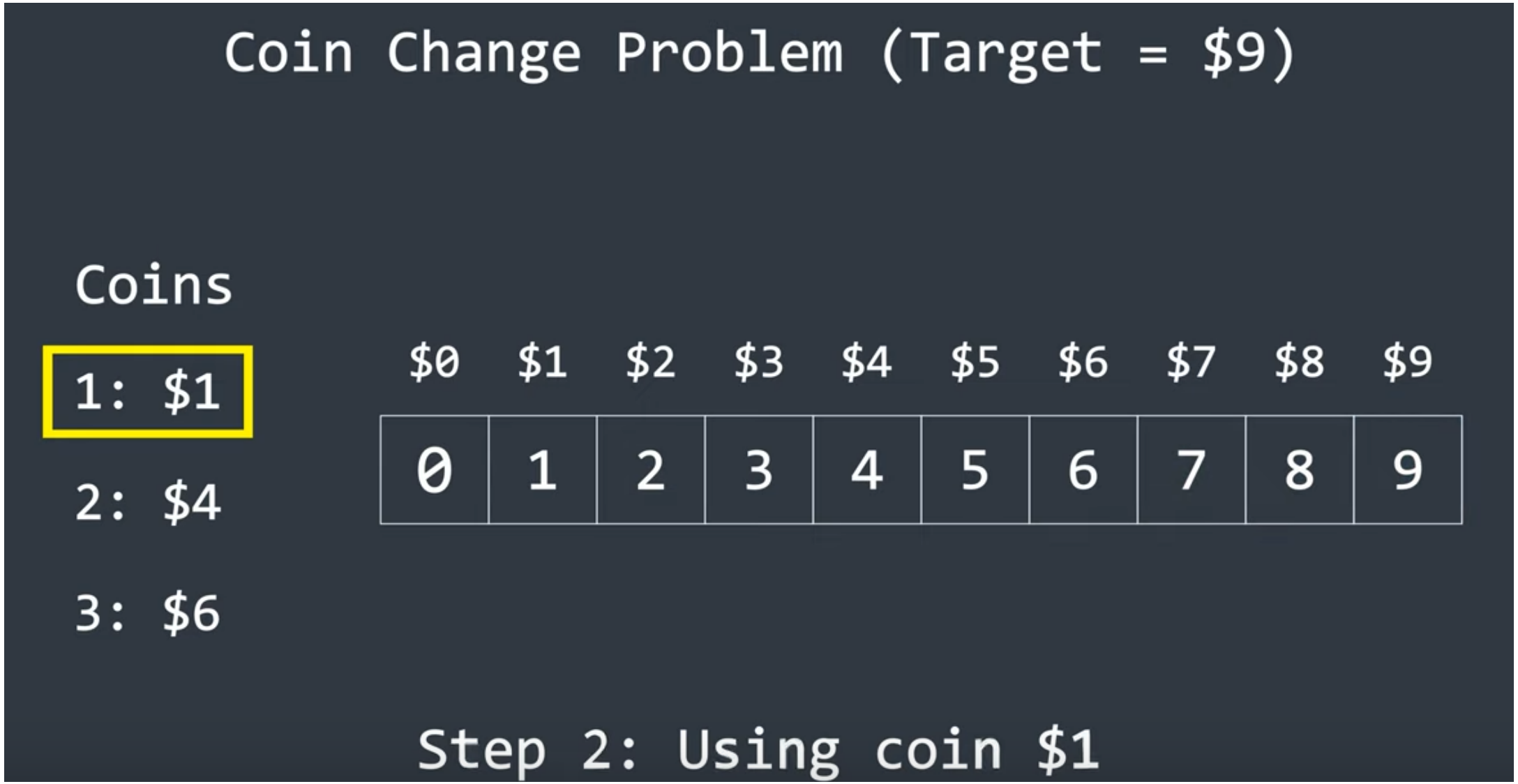
```
coins = [1, 4, 6]    infinite supply

Amount = 9
```

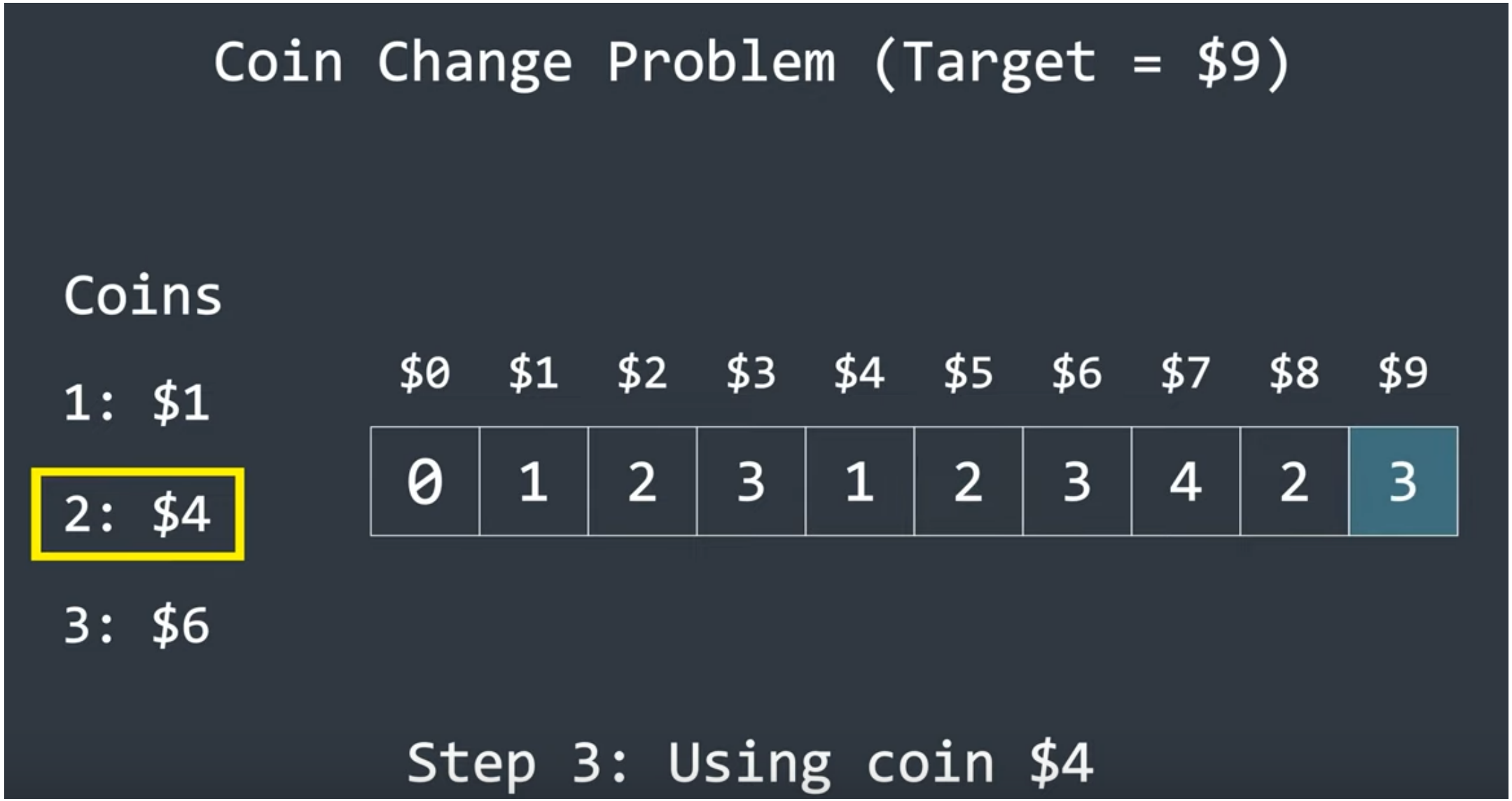
https://www.youtube.com/watch?v=KnWorqyDSLA



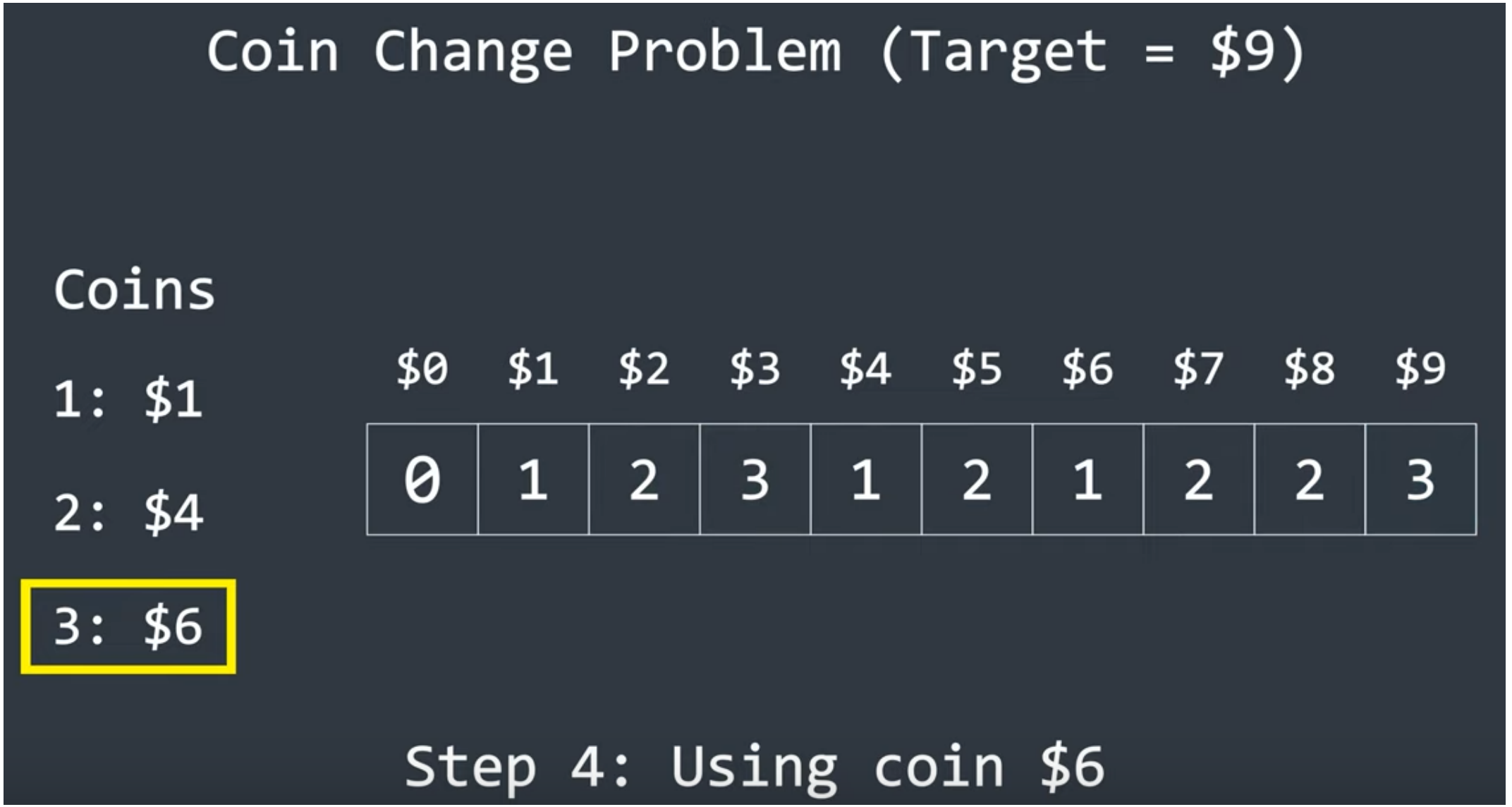
https://www.youtube.com/watch?v=KnWorqyDSLA



https://www.youtube.com/watch?v=KnWorqyDSLA



https://www.youtube.com/watch?v=KnWorqyDSLA



https://www.youtube.com/watch?v=KnWorqyDSLA

```python
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
```

https://www.youtube.com/watch?v=KnWorqyDSLA

```python
def coin_change(coins, amount):

    dp = [float('inf')] * (amount + 1)

    dp[0] = 0

    for coin in coins:
        for current_amount in range(coin, amount + 1):
```

https://www.youtube.com/watch?v=KnWorqyDSLA

```python
def coin_change(coins, amount):

    dp = [float('inf')] * (amount + 1)

    dp[0] = 0

    for coin in coins:
        for current_amount in range(coin, amount + 1):

            dp[current_amount] = min(dp[current_amount],
                                     dp[current_amount - coin] + 1)
```

https://www.youtube.com/watch?v=KnWorqyDSLA

```python
def coin_change(coins, amount):

    dp = [float('inf')] * (amount + 1)

    dp[0] = 0

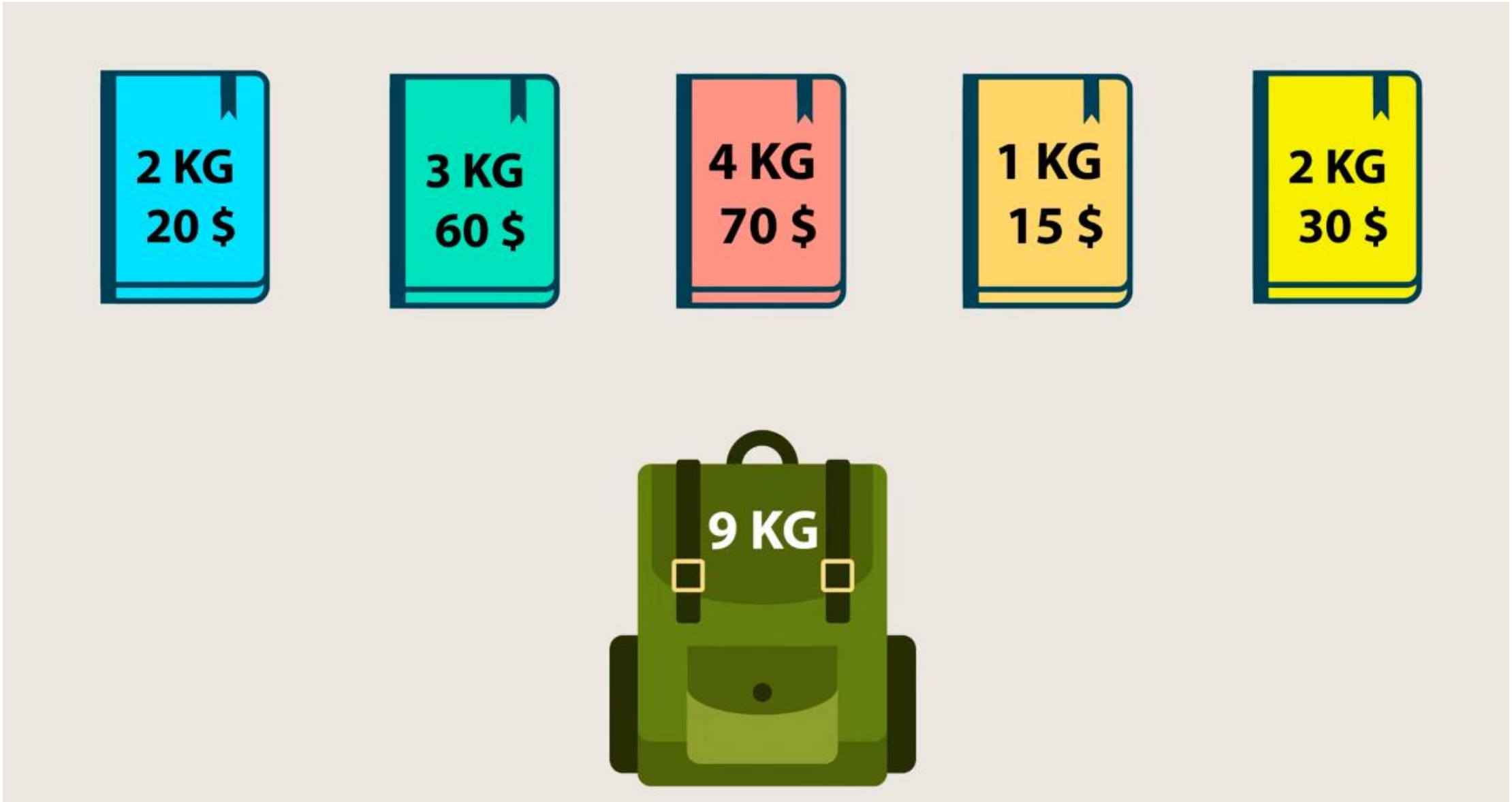
    for coin in coins:
        for current_amount in range(coin, amount + 1):

            dp[current_amount] = min(dp[current_amount],
                                     dp[current_amount - coin] + 1)

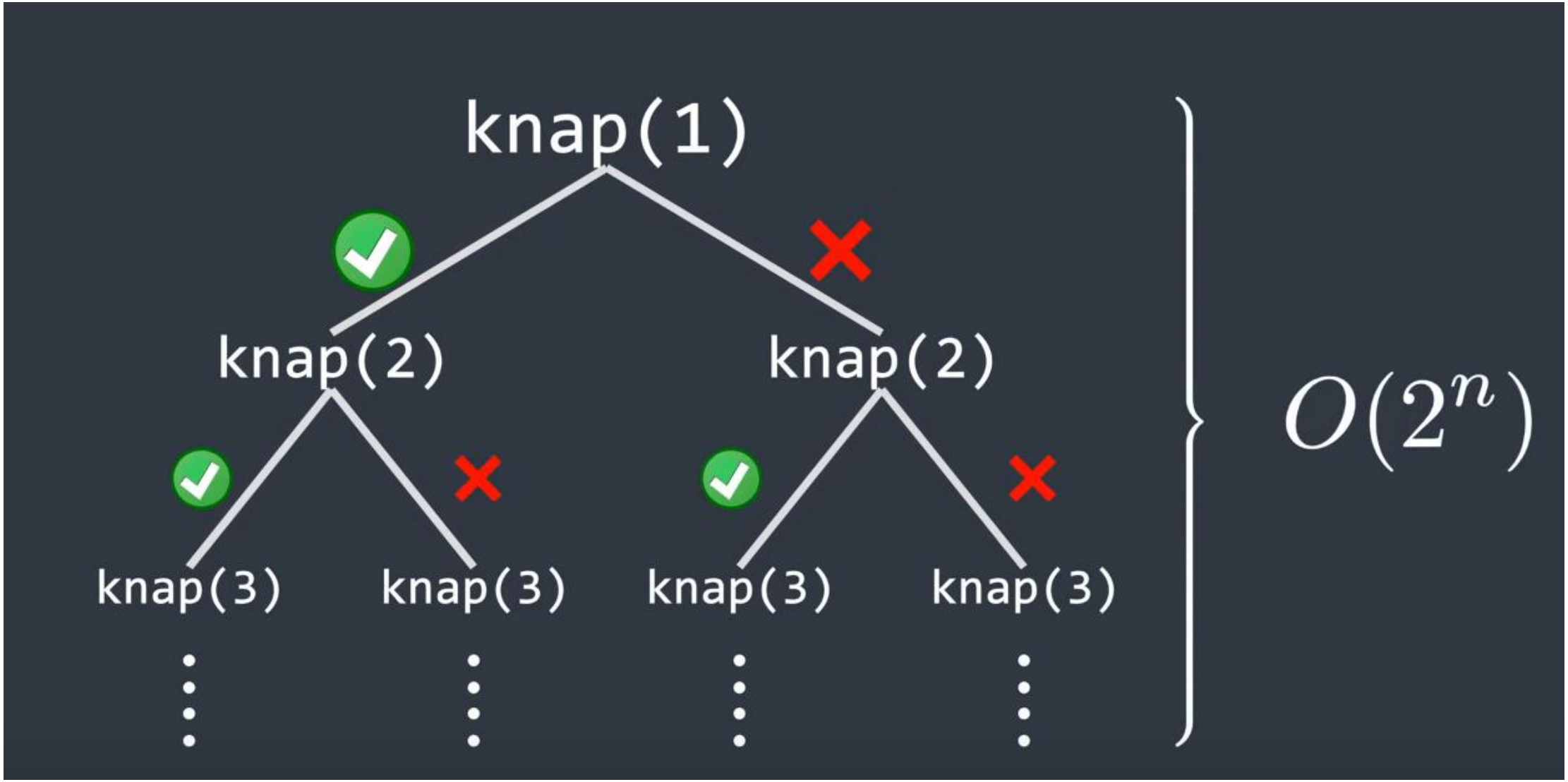
    return dp[amount] if dp[amount] != float('inf') else -1
```

Knapsack problem

https://www.youtube.com/watch?v=qxWu-SeAqe4

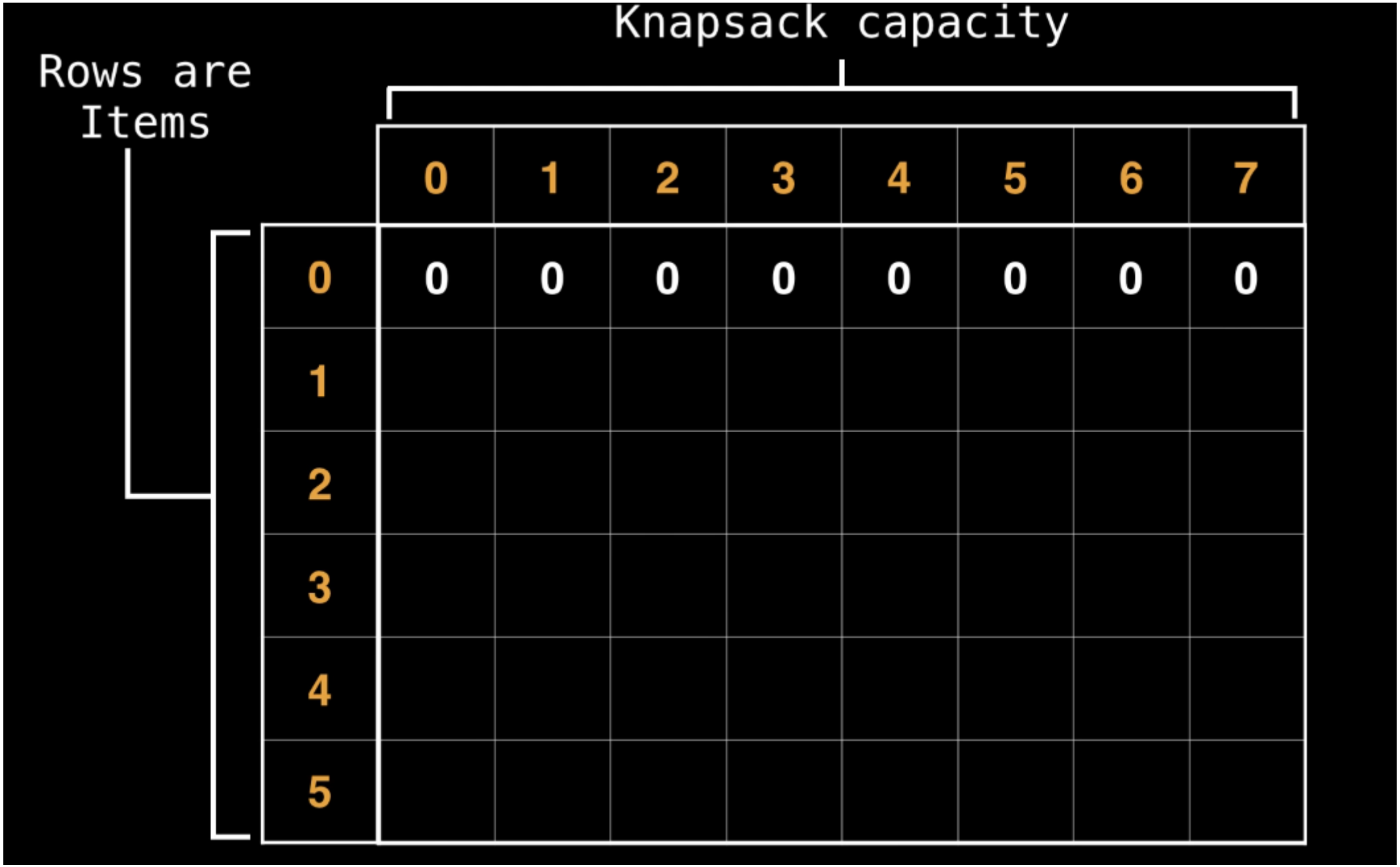


https://www.youtube.com/watch?v=qxWu-SeAqe4

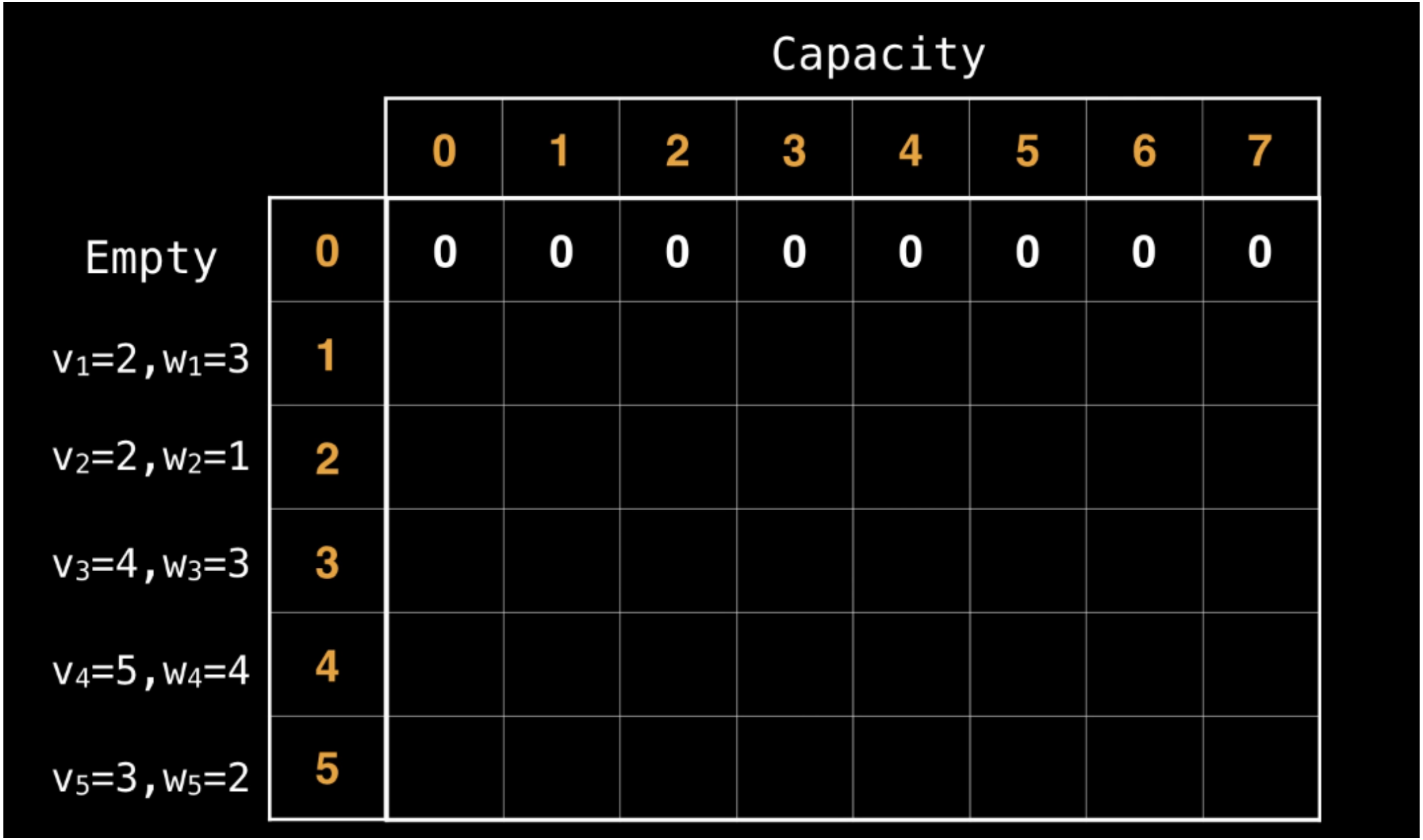



It’s drawing time!

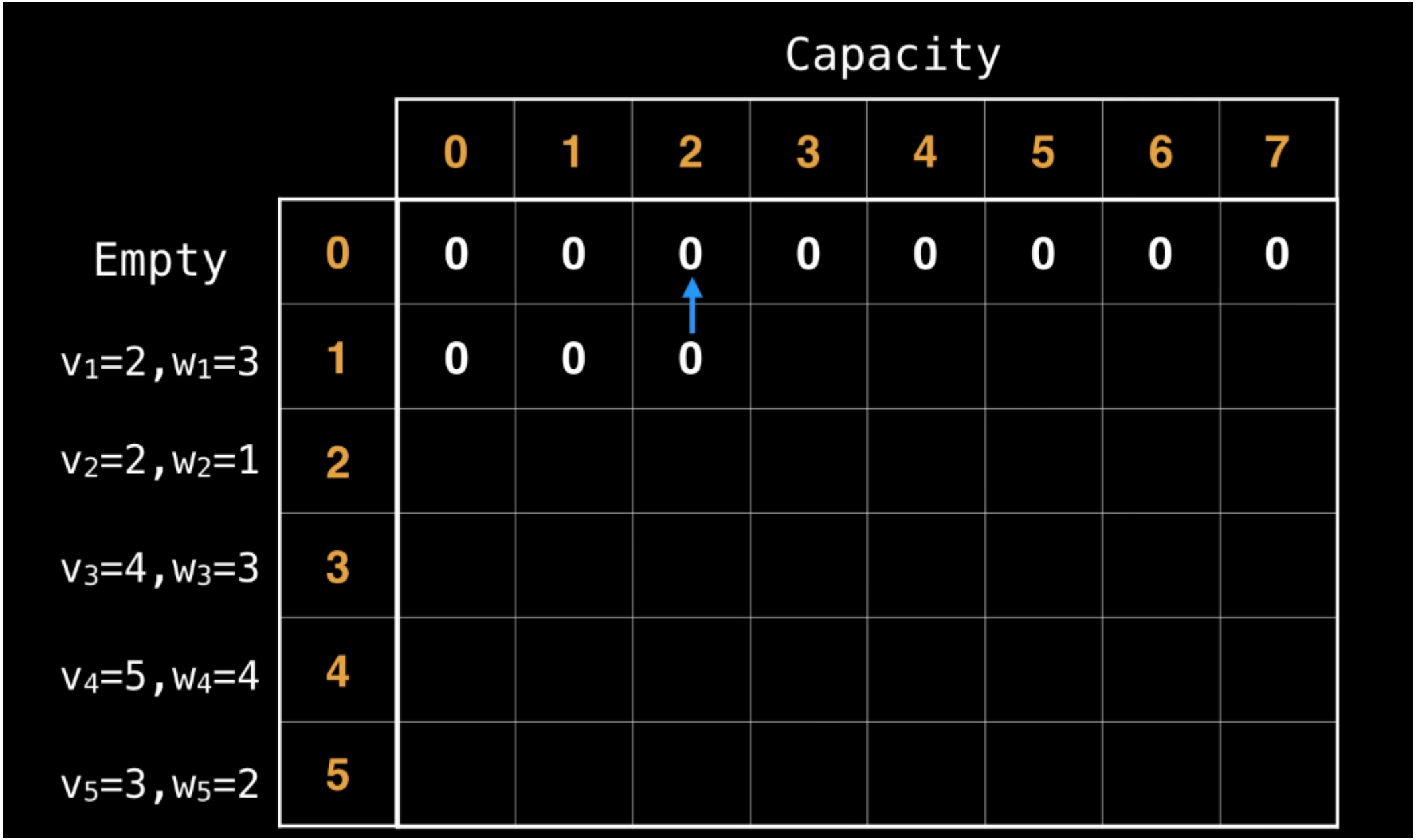
https://www.youtube.com/watch?v=cJ21moQpofY



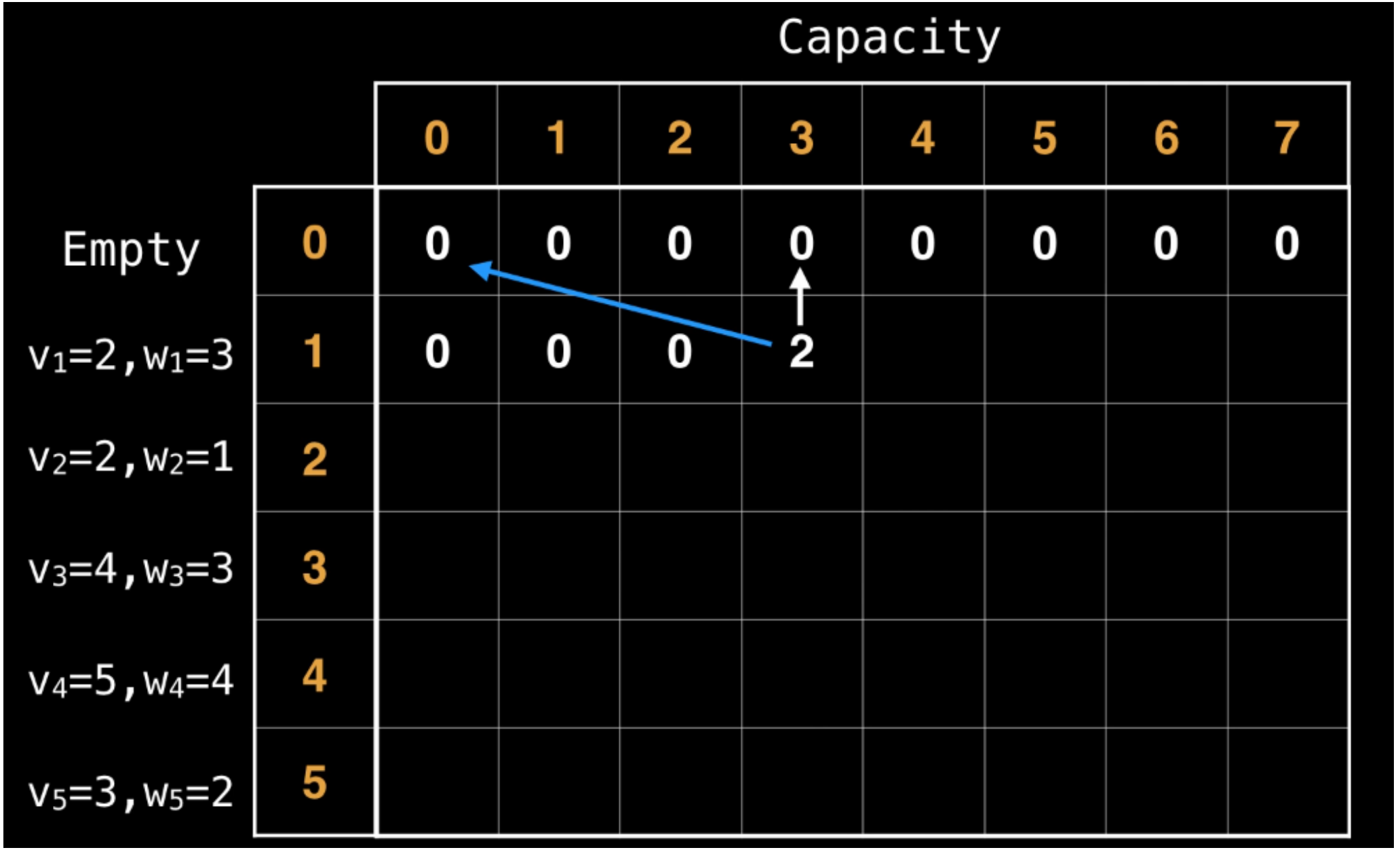
https://www.youtube.com/watch?v=cJ21moQpofY



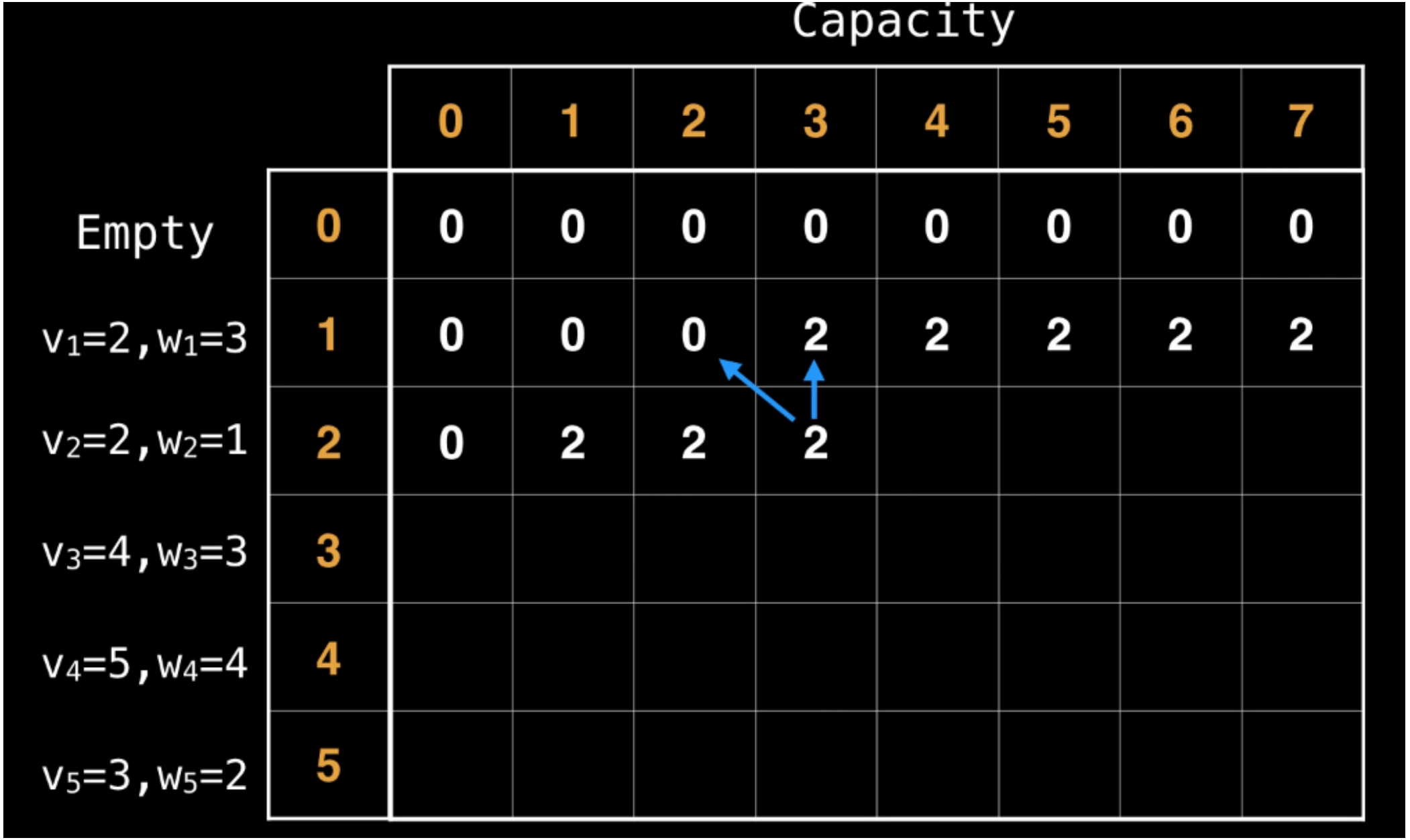
https://www.youtube.com/watch?v=cJ21moQpofY



https://www.youtube.com/watch?v=cJ21moQpofY



https://www.youtube.com/watch?v=cJ21moQpofY



https://www.youtube.com/watch?v=cJ21moQpofY

		0	1	2	3	4	5	6	7
Empty	0	0	0	0	0	0	0	0	0
$v_1=2, w_1=3$	1	0	0	0	2	2	2	2	2
$v_2=2, w_2=1$	2	0	2	2	2	4	4	4	4
$v_3=4, w_3=3$	3	0	2	2	4	6	6	6	8
$v_4=5, w_4=4$	4	0	2	2	4	6	7	7	9
$v_5=3, w_5=2$	5	0	2	3	5	6	7	9	10

https://github.com/ByteQuest0/Implemention_codes/blob/main/Dynamic%20Programming/knapsack.py

```python
def knapsack(weights, profits, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
```

https://github.com/ByteQuest0/Implemention_codes/blob/main/Dynamic%20Programming/knapsack.py

```python
def knapsack(weights, profits, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    # Build DP table
    for i in range(1, n + 1):  # Items
        for w in range(capacity + 1):  # Capacity from 0 to W
```

https://github.com/ByteQuest0/Implemention_codes/blob/main/Dynamic%20Programming/knapsack.py

```python
def knapsack(weights, profits, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    # Build DP table
    for i in range(1, n + 1):  # Items
        for w in range(capacity + 1):  # Capacity from 0 to W
            if weights[i-1] > w:  # If item is too heavy, exclude it
                dp[i][w] = dp[i-1][w]
```

https://github.com/ByteQuest0/Implemention_codes/blob/main/Dynamic%20Programming/knapsack.py

```python
def knapsack(weights, profits, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

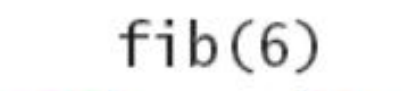
    # Build DP table
    for i in range(1, n + 1):  # Items
        for w in range(capacity + 1):  # Capacity from 0 to W
            if weights[i-1] > w:  # If item is too heavy, exclude it
                dp[i][w] = dp[i-1][w]
            else:
                dp[i][w] = max(dp[i-1][w], profits[i-1] + dp[i-1][w - weights[i-1]])

```

Appendix

What does recursion *really* look like?

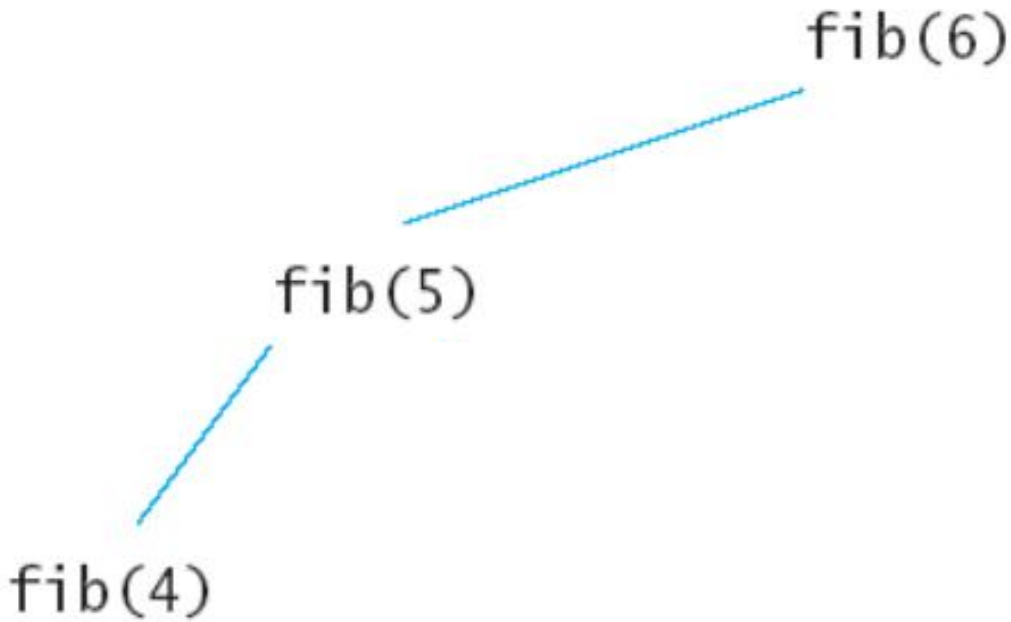
https://www.artofcse.com/learning/introduction-to-dynamic-programming



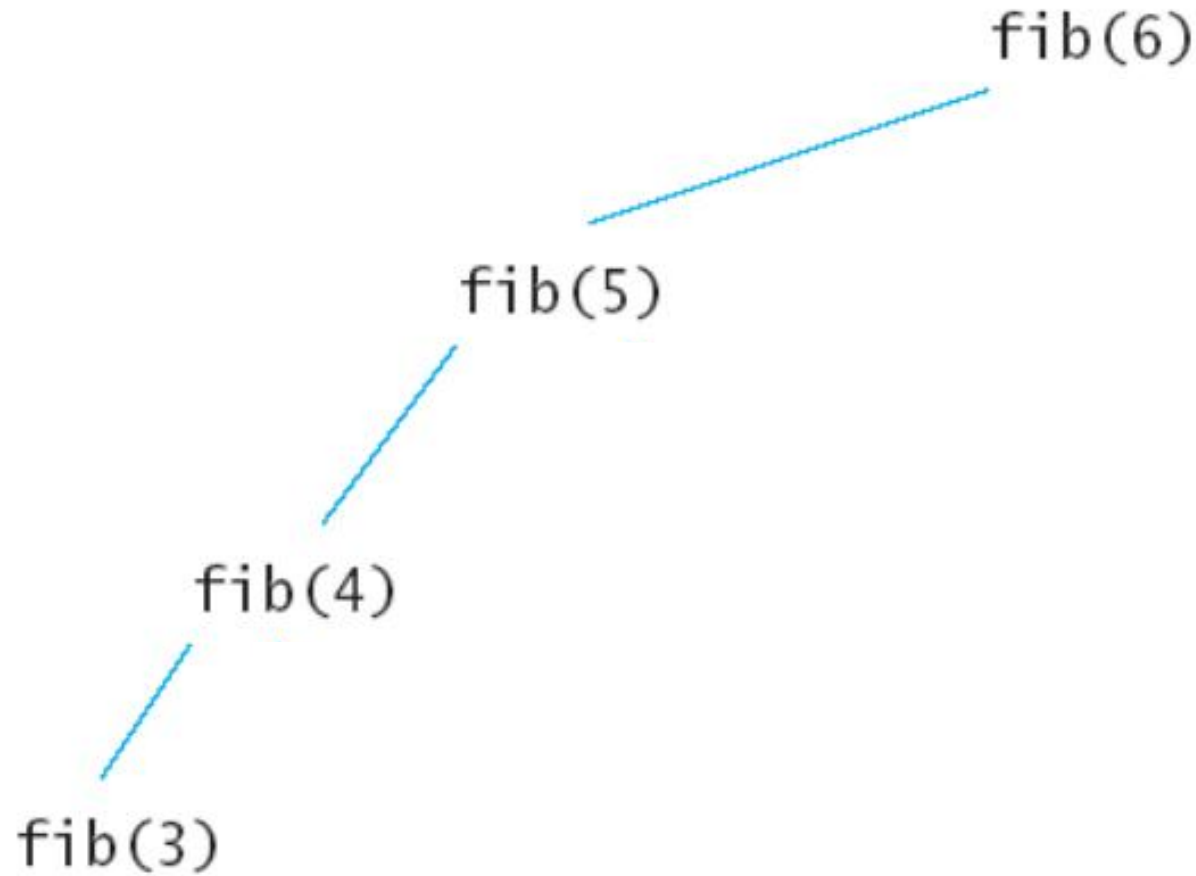
https://www.artofcse.com/learning/introduction-to-dynamic-programming



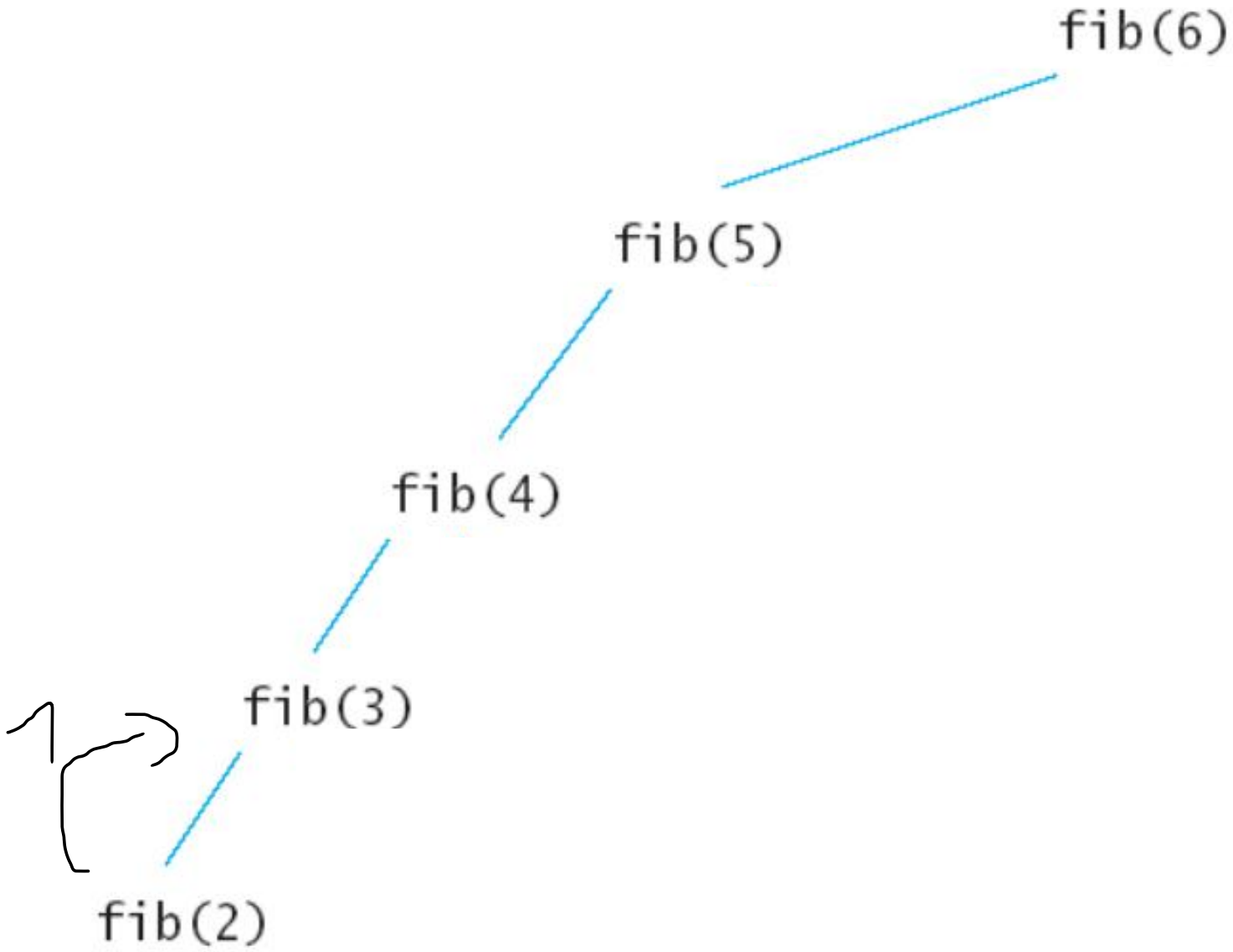
https://www.artofcse.com/learning/introduction-to-dynamic-programming



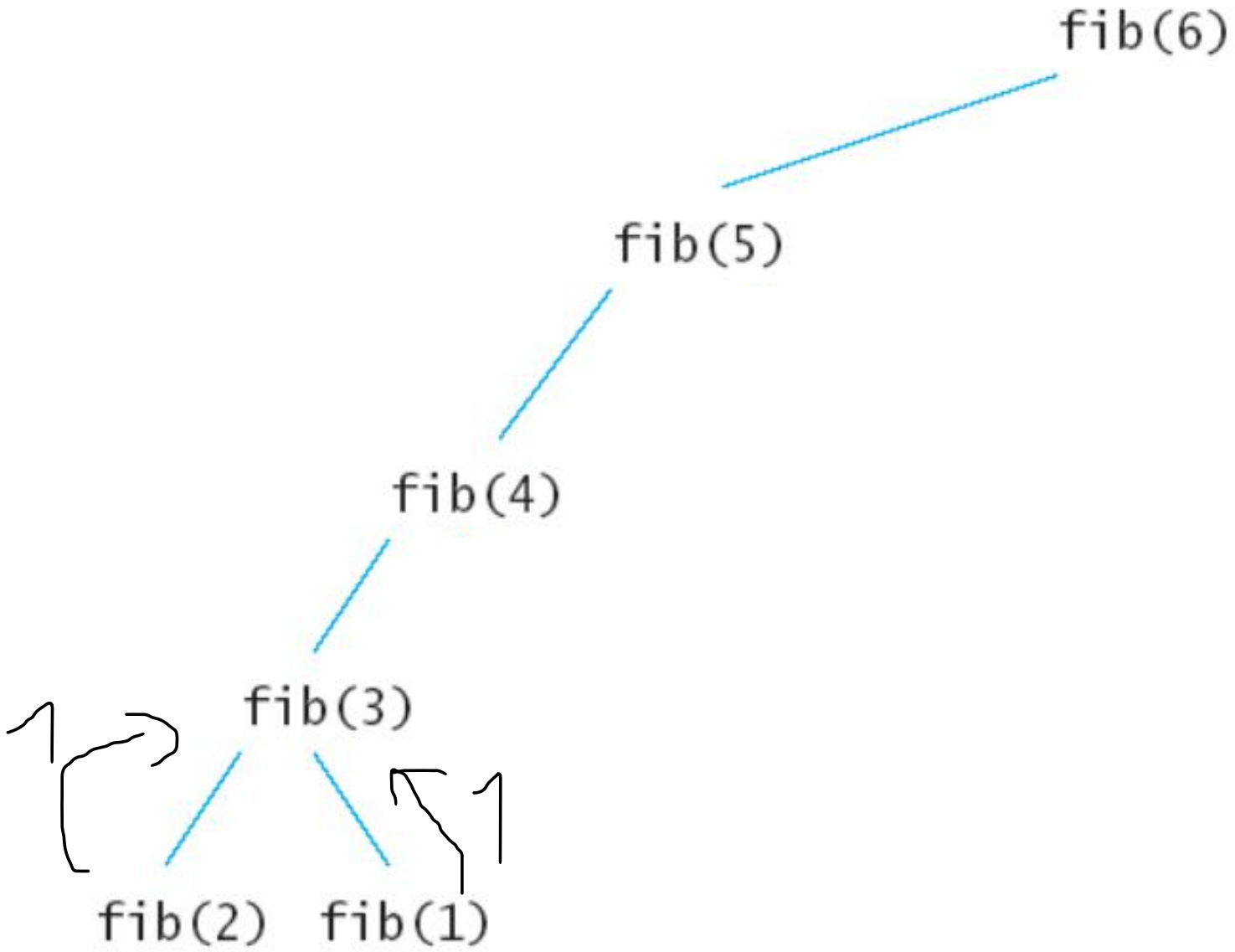
https://www.artofcse.com/learning/introduction-to-dynamic-programming



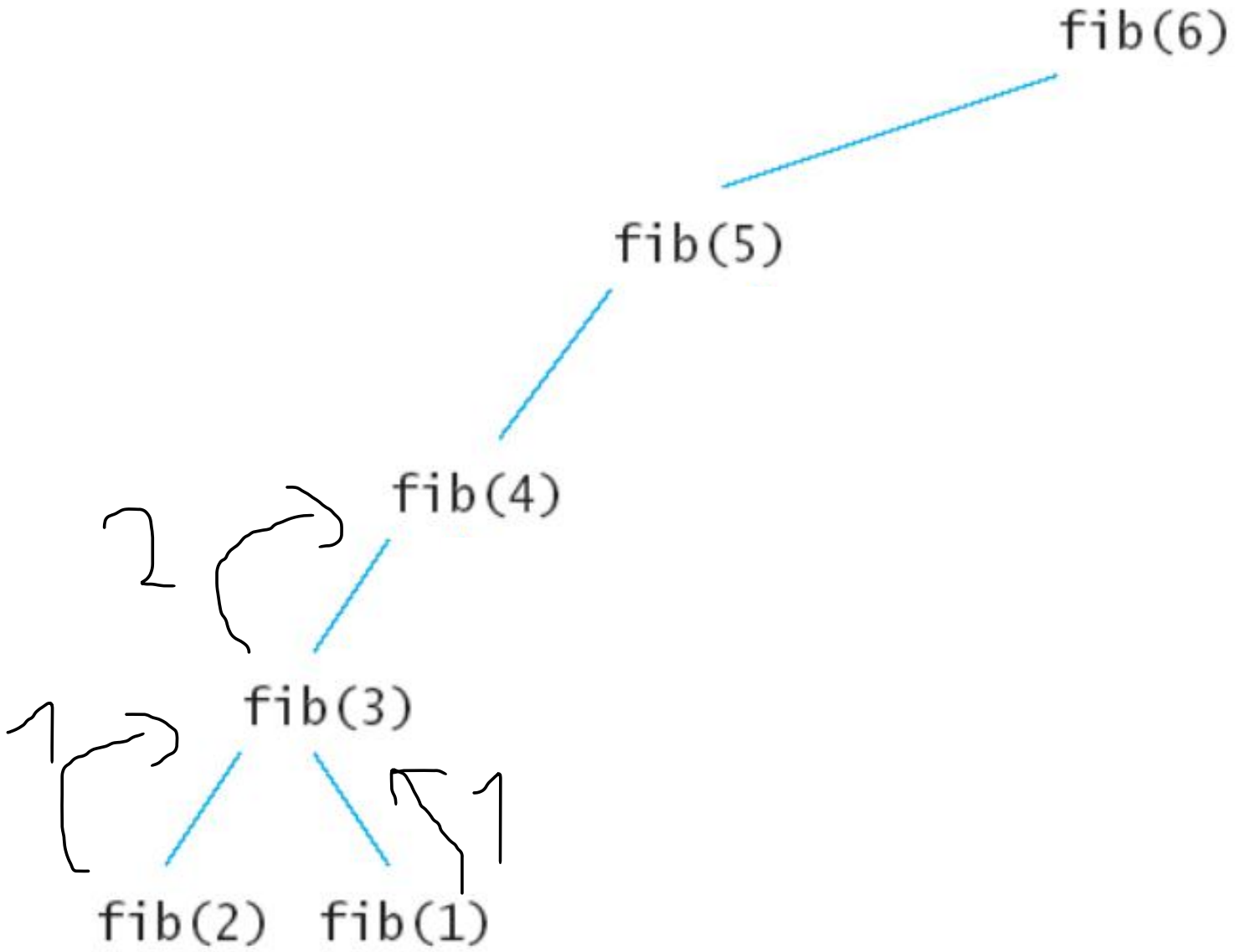
https://www.artofcse.com/learning/introduction-to-dynamic-programming



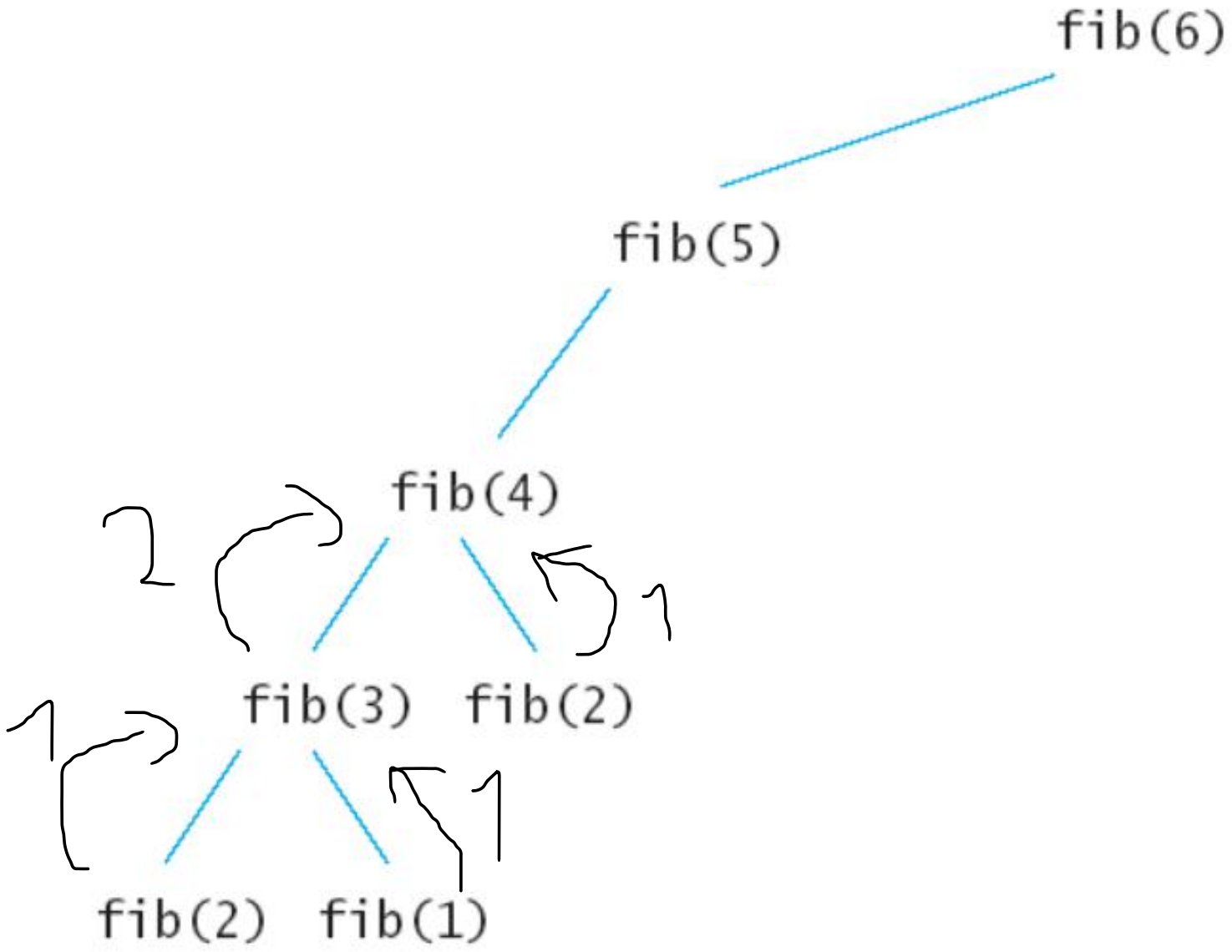
https://www.artofcse.com/learning/introduction-to-dynamic-programming



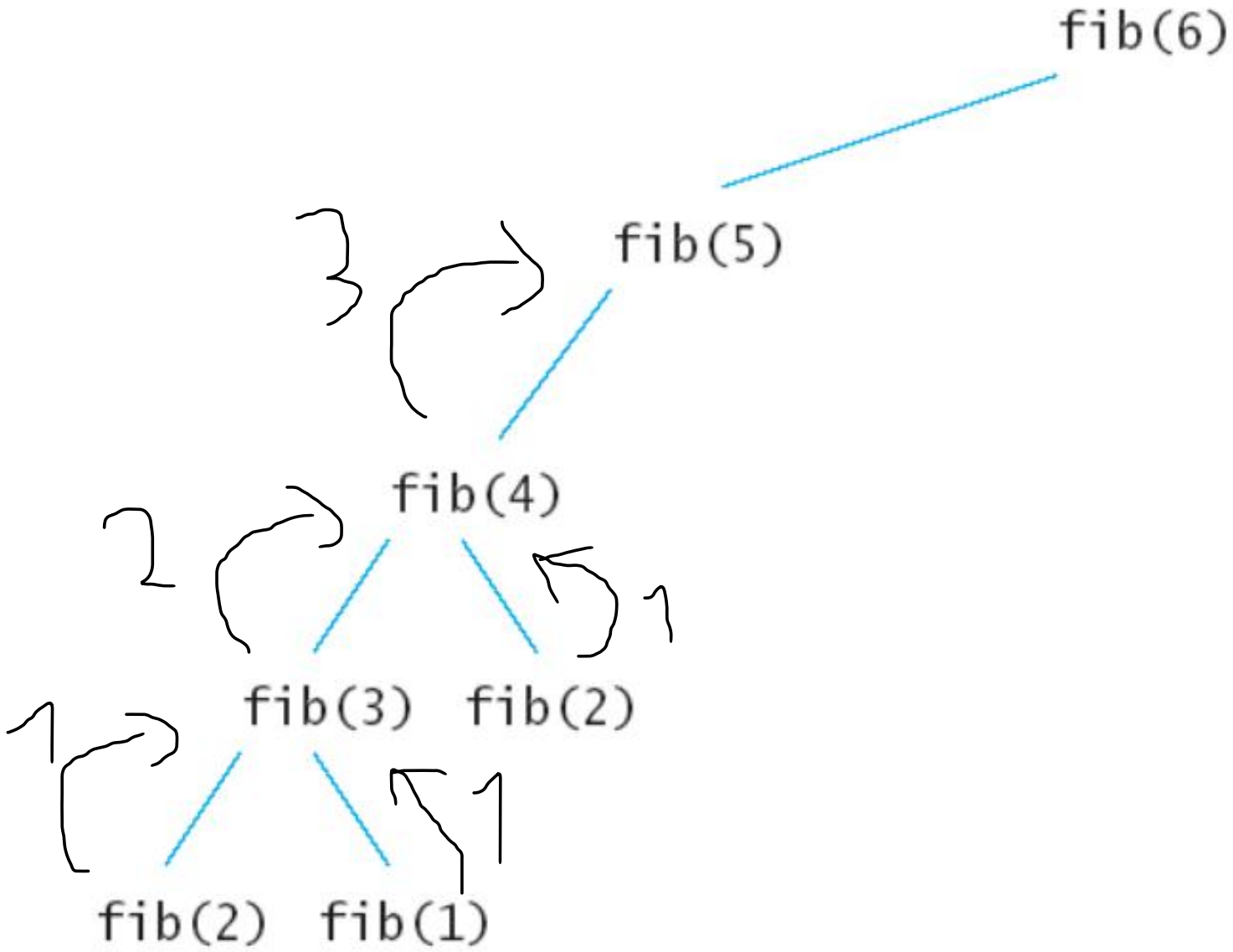
https://www.artofcse.com/learning/introduction-to-dynamic-programming



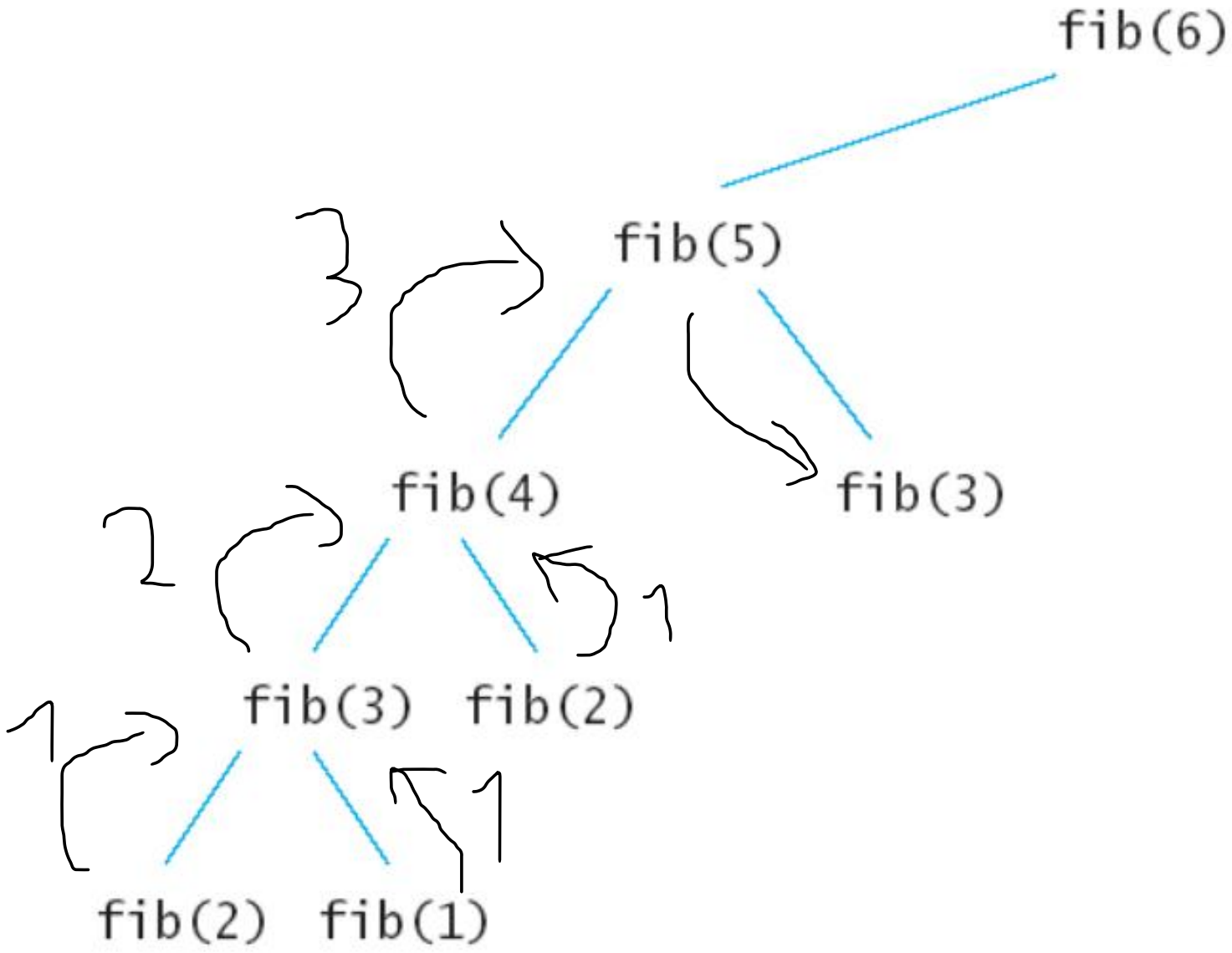
https://www.artofcse.com/learning/introduction-to-dynamic-programming



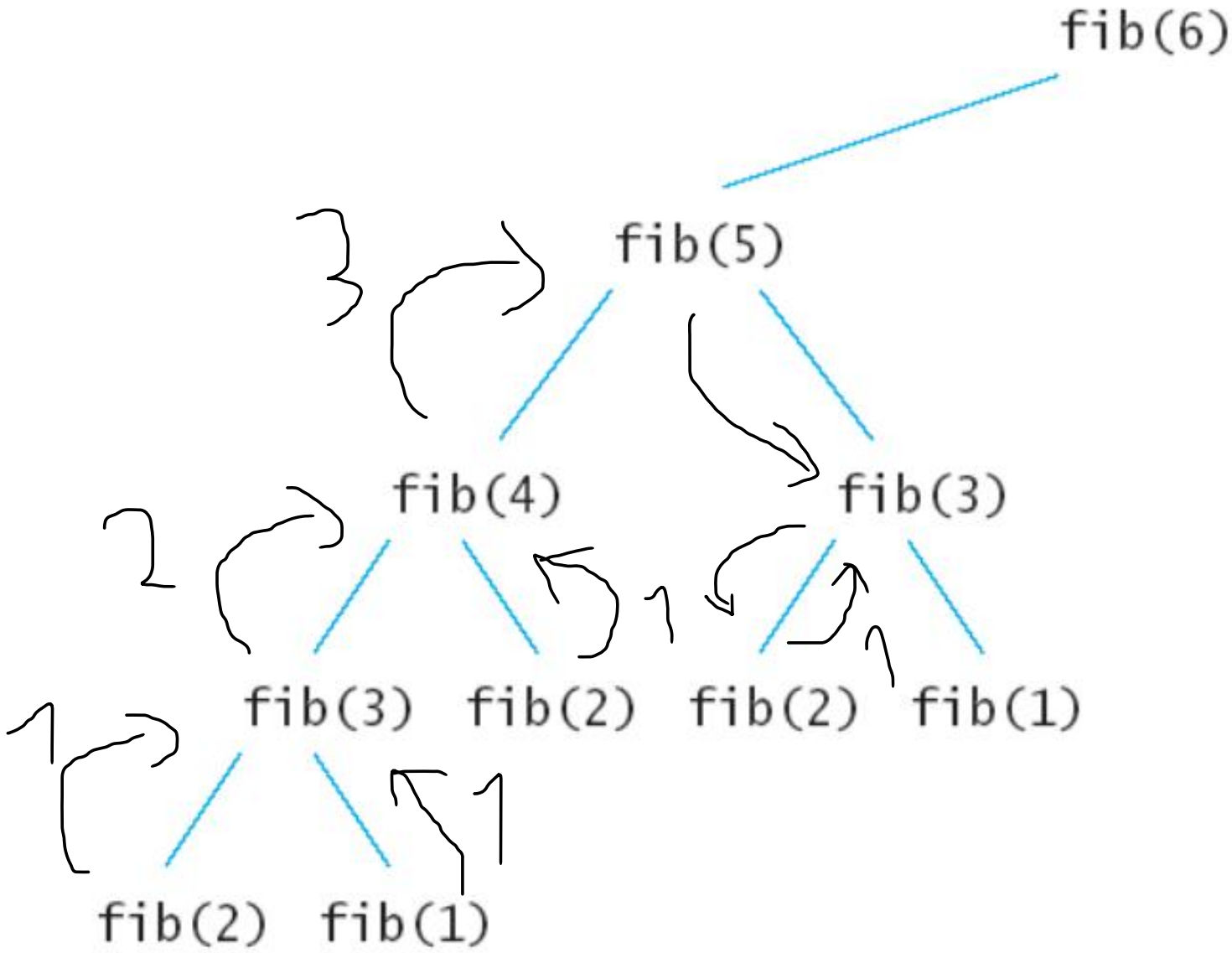
https://www.artofcse.com/learning/introduction-to-dynamic-programming



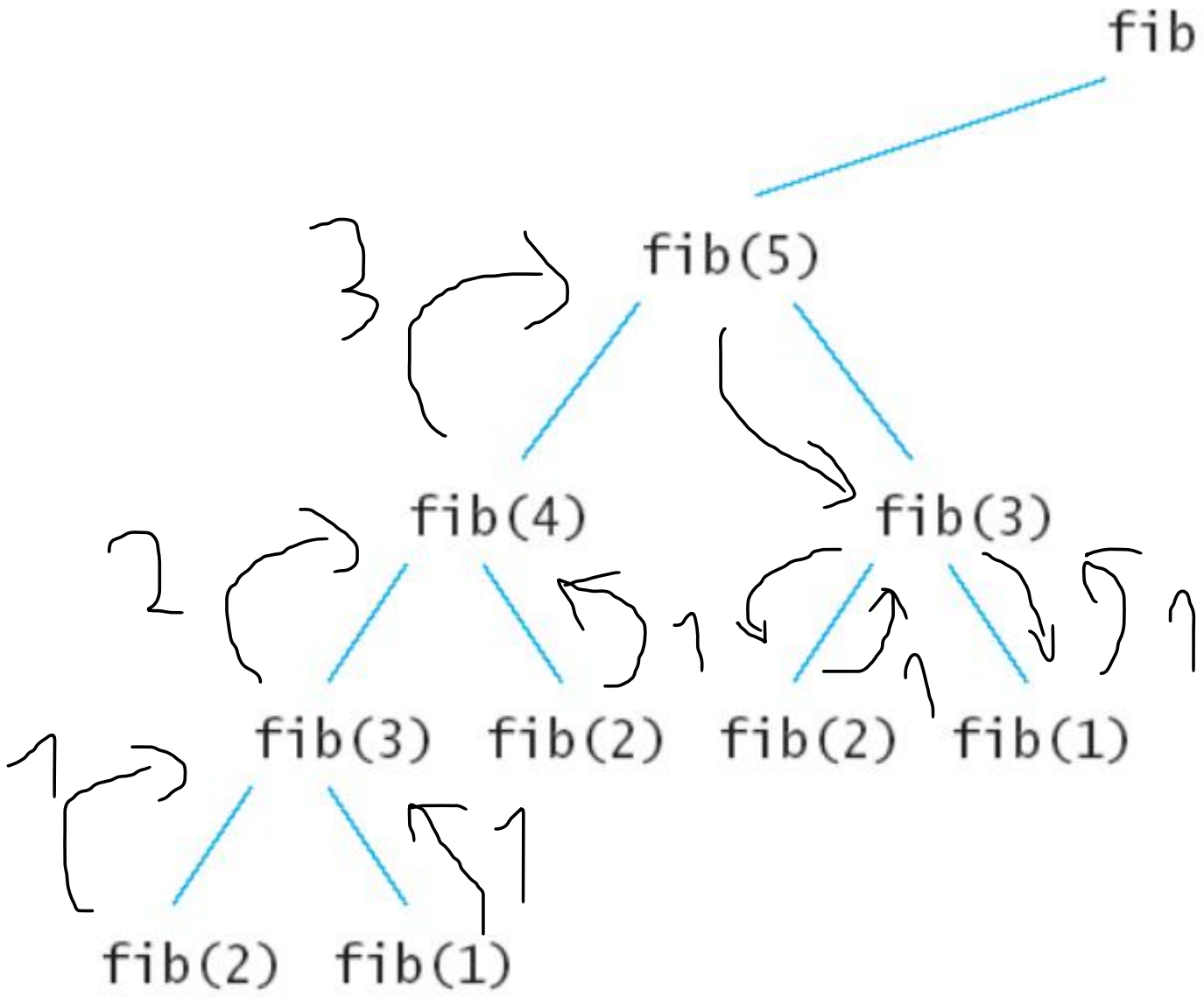
https://www.artofcse.com/learning/introduction-to-dynamic-programming



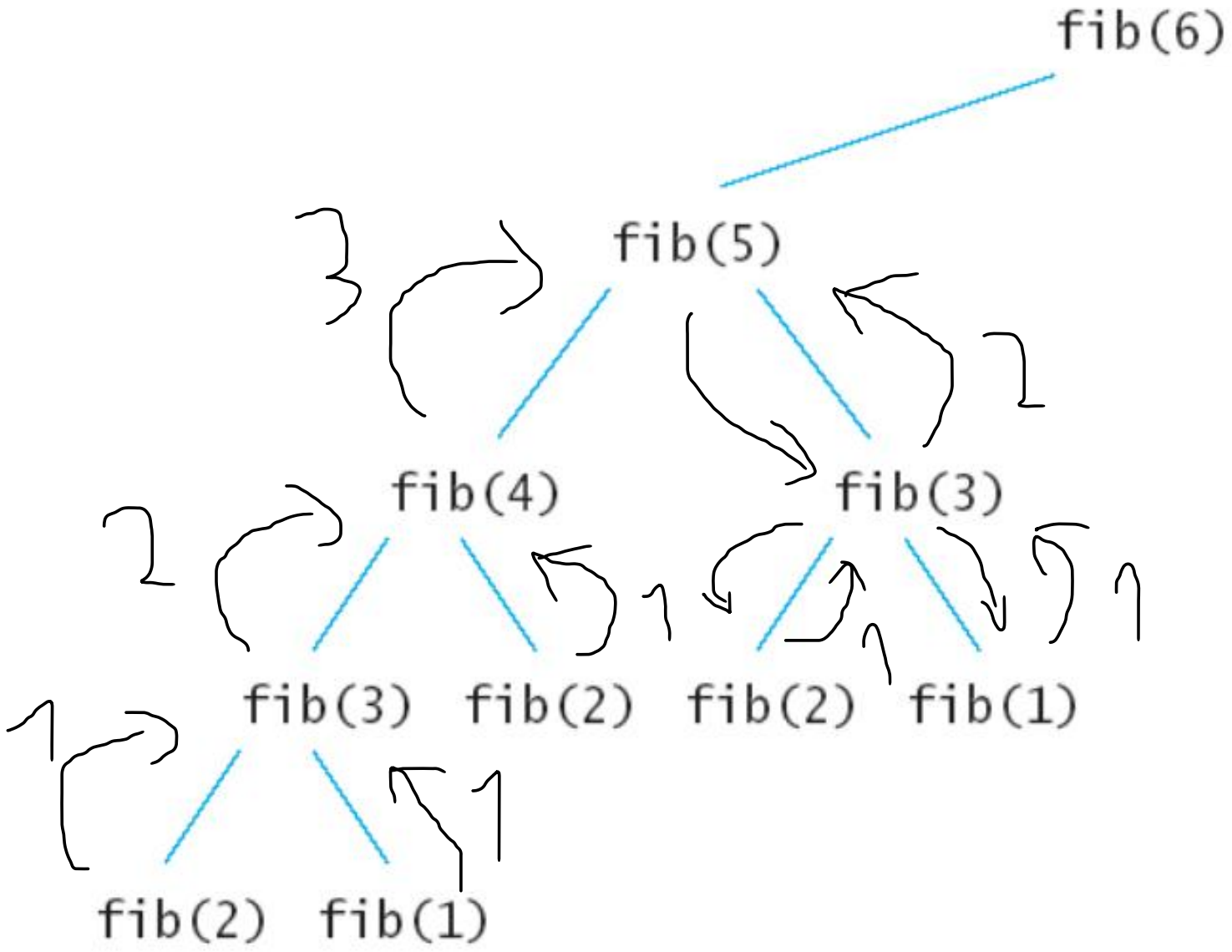
https://www.artofcse.com/learning/introduction-to-dynamic-programming



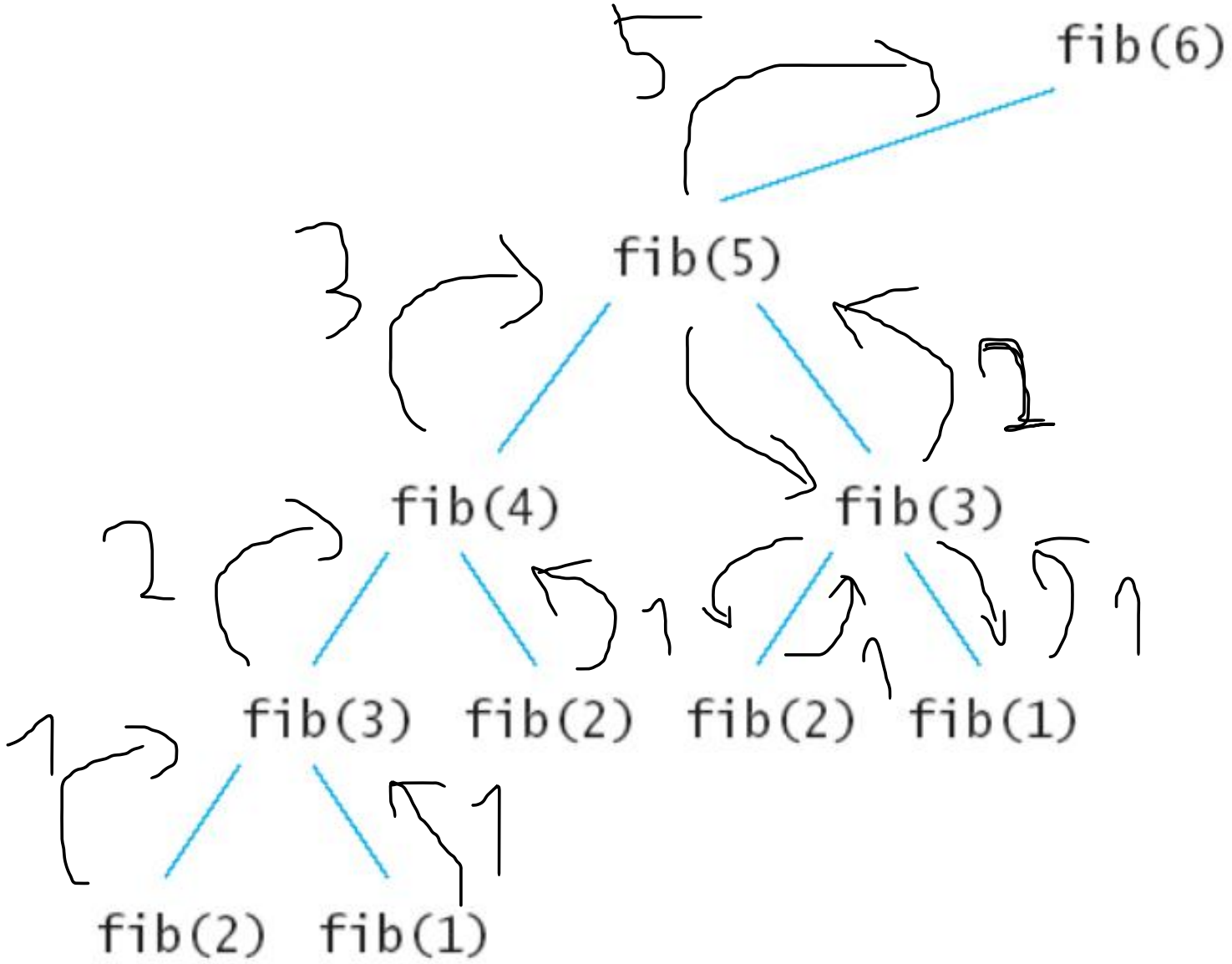
https://www.artofcse.com/learning/introduction-to-dynamic-programming



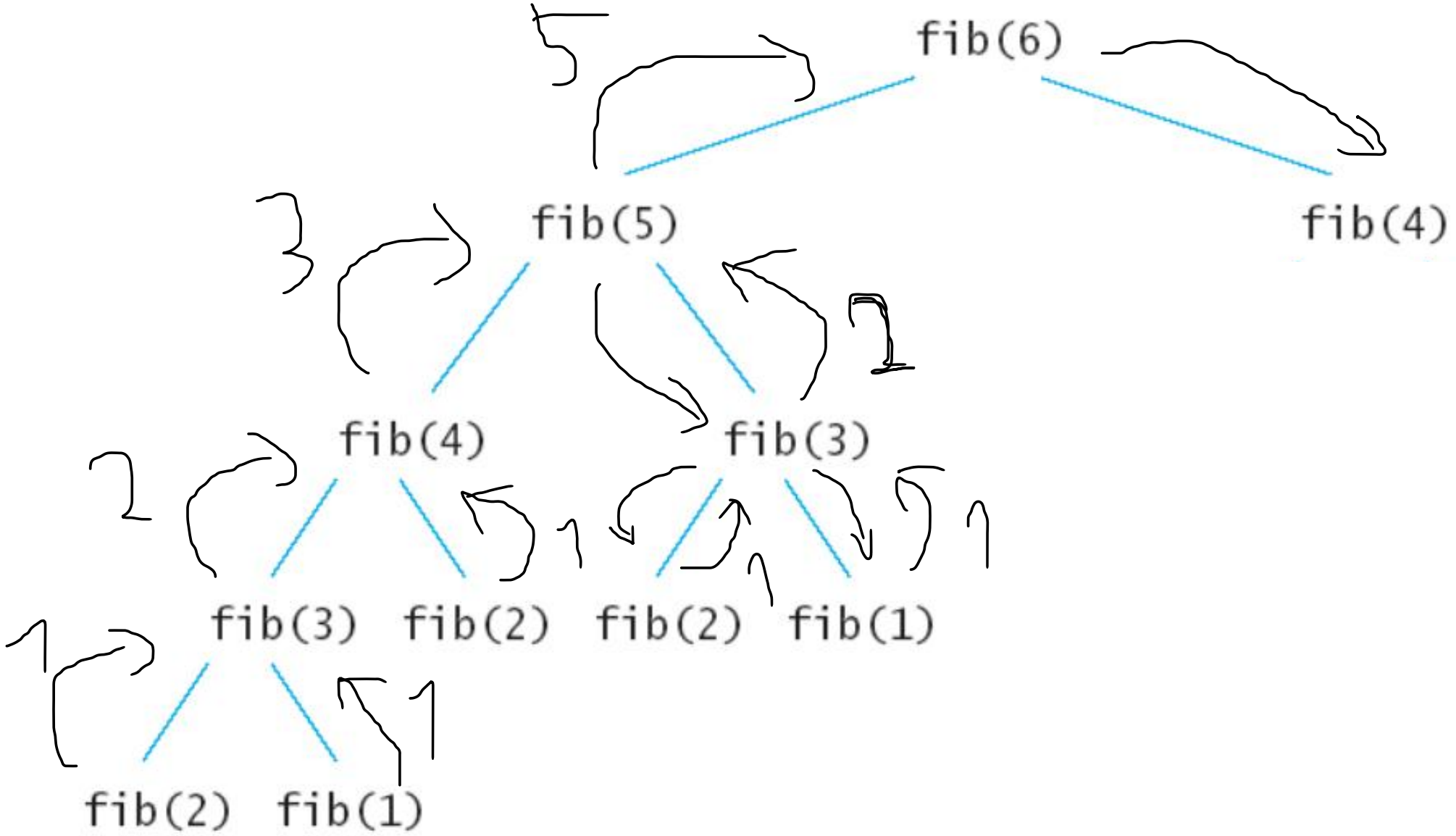
https://www.artofcse.com/learning/introduction-to-dynamic-programming



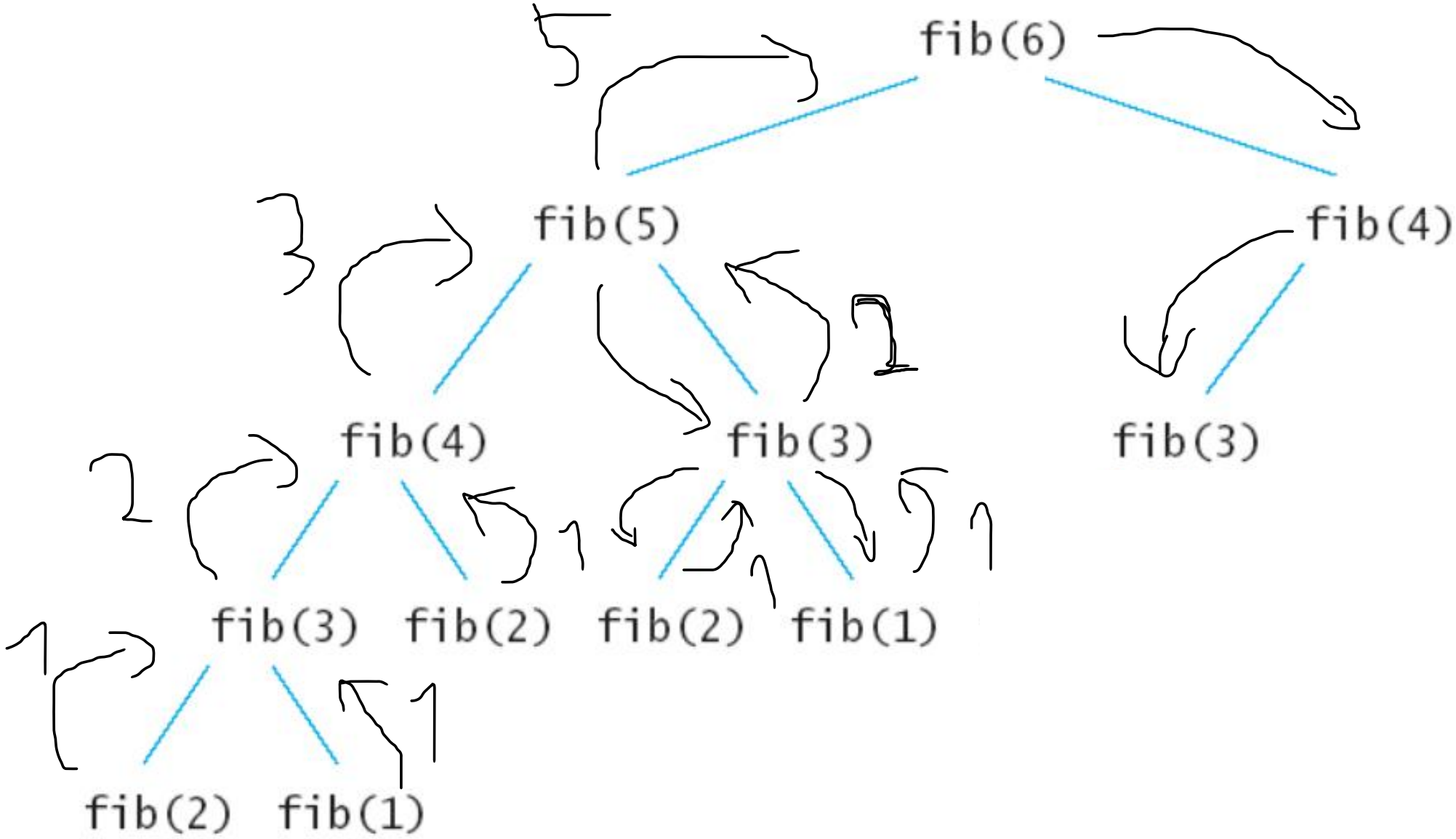
https://www.artofcse.com/learning/introduction-to-dynamic-programming



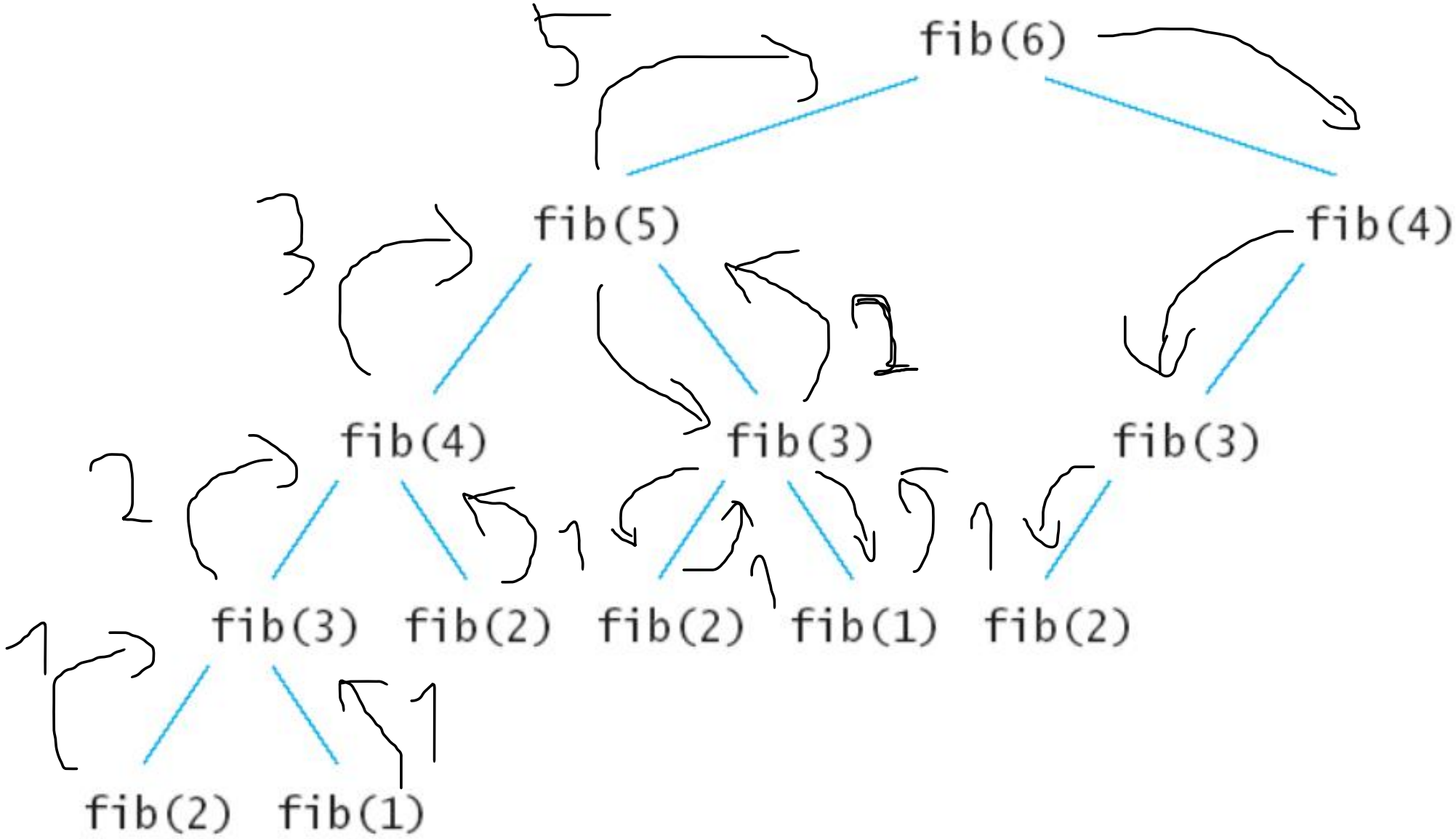
https://www.artofcse.com/learning/introduction-to-dynamic-programming



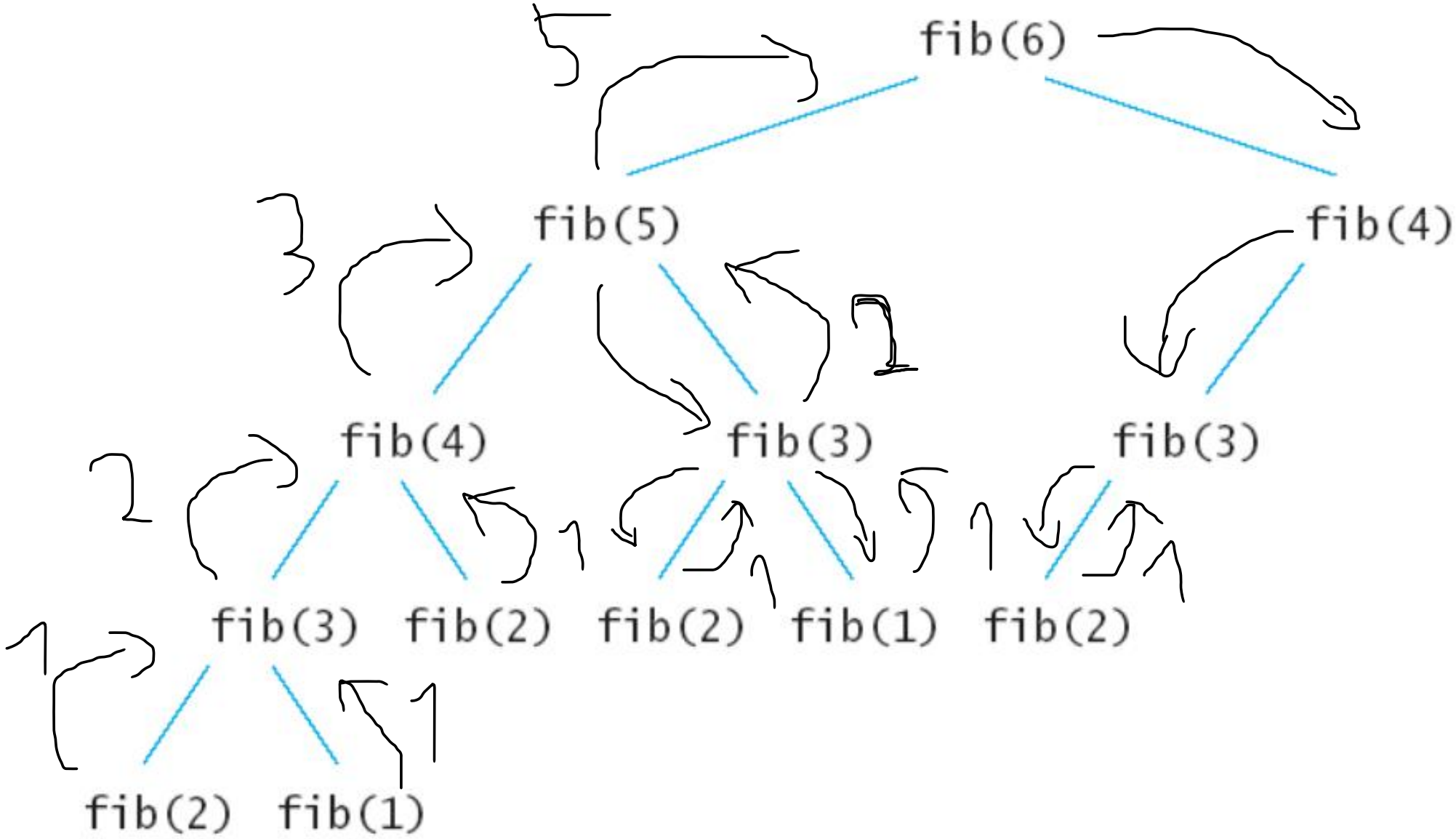
https://www.artofcse.com/learning/introduction-to-dynamic-programming



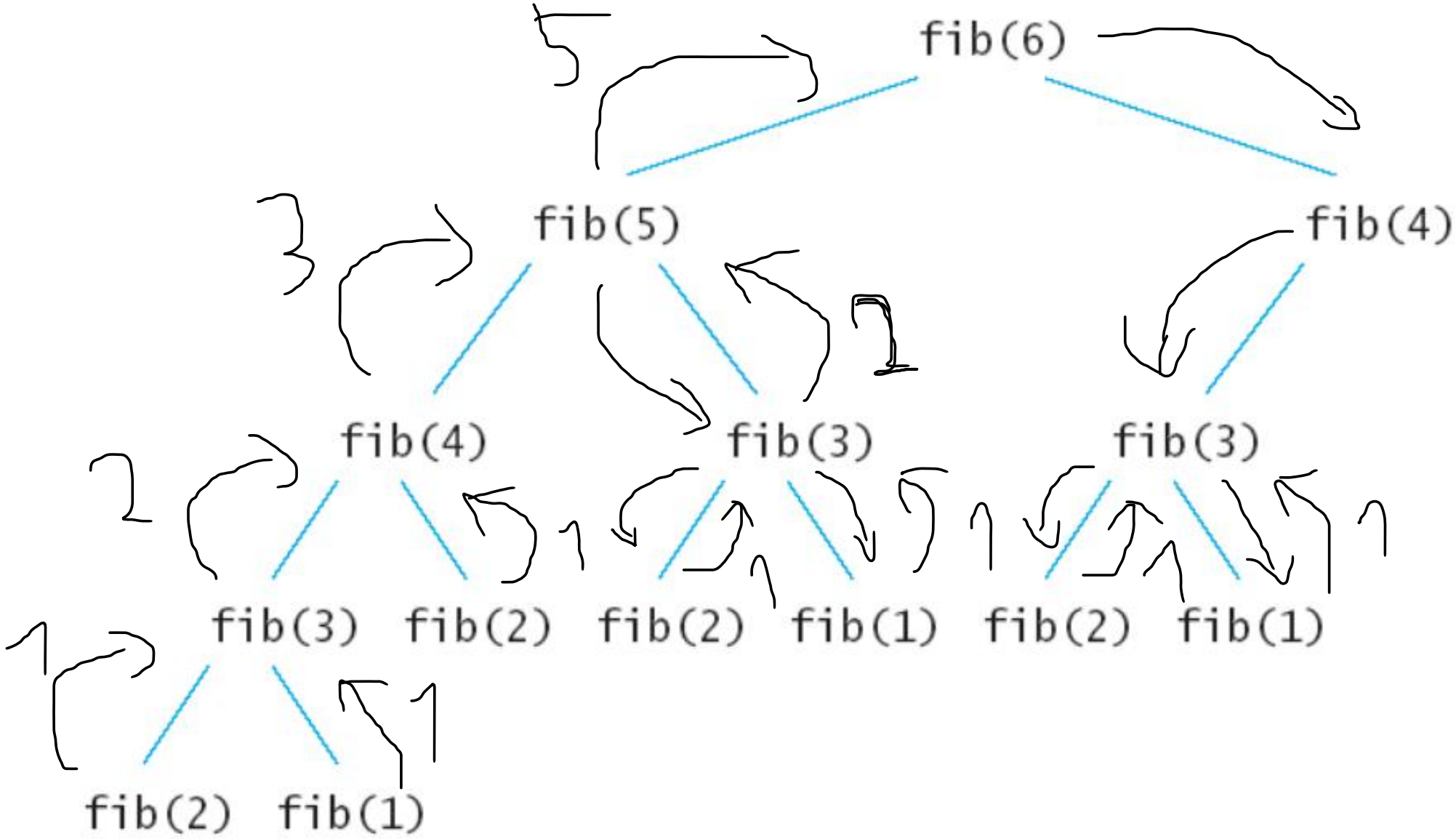
https://www.artofcse.com/learning/introduction-to-dynamic-programming



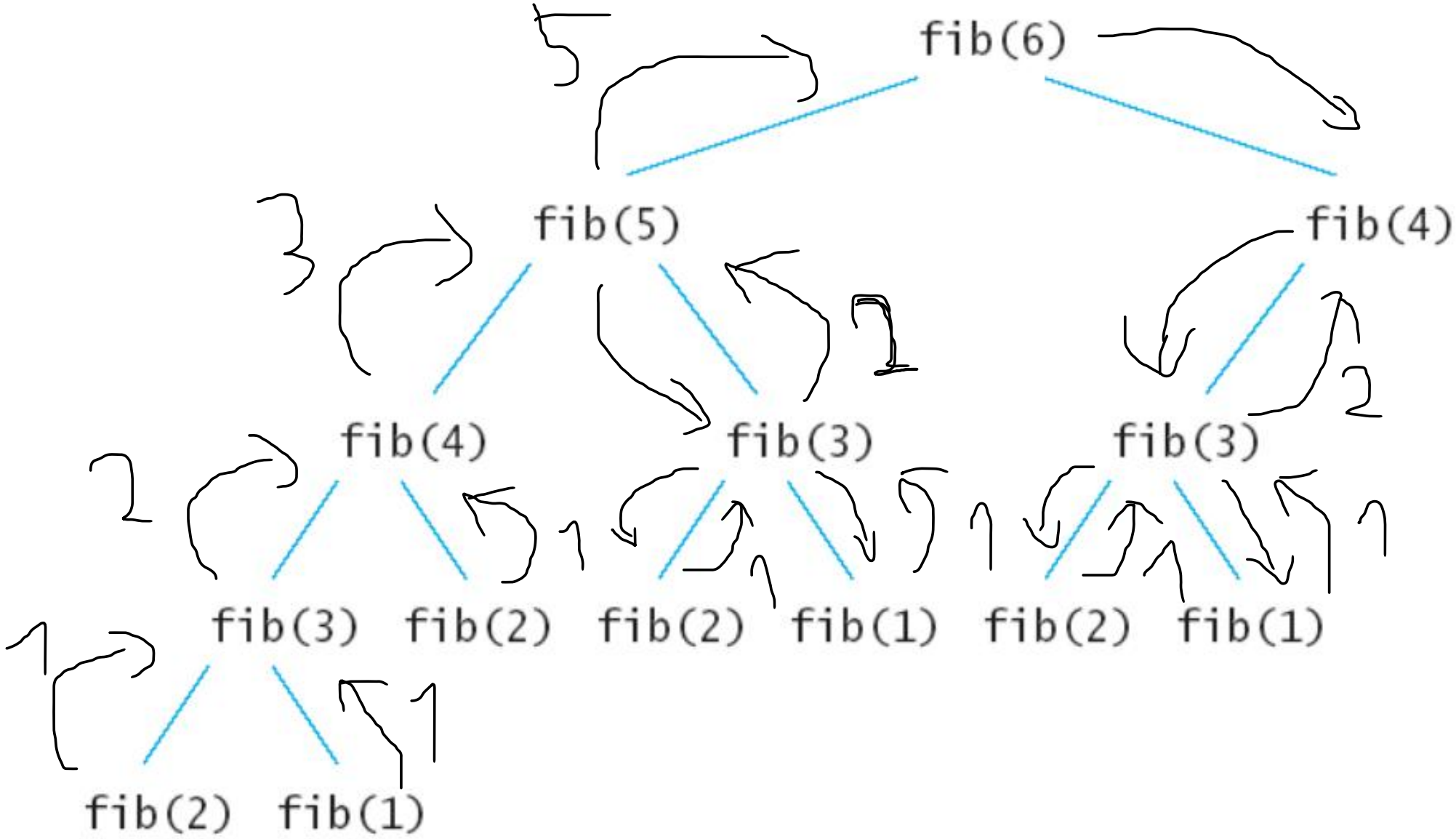
https://www.artofcse.com/learning/introduction-to-dynamic-programming



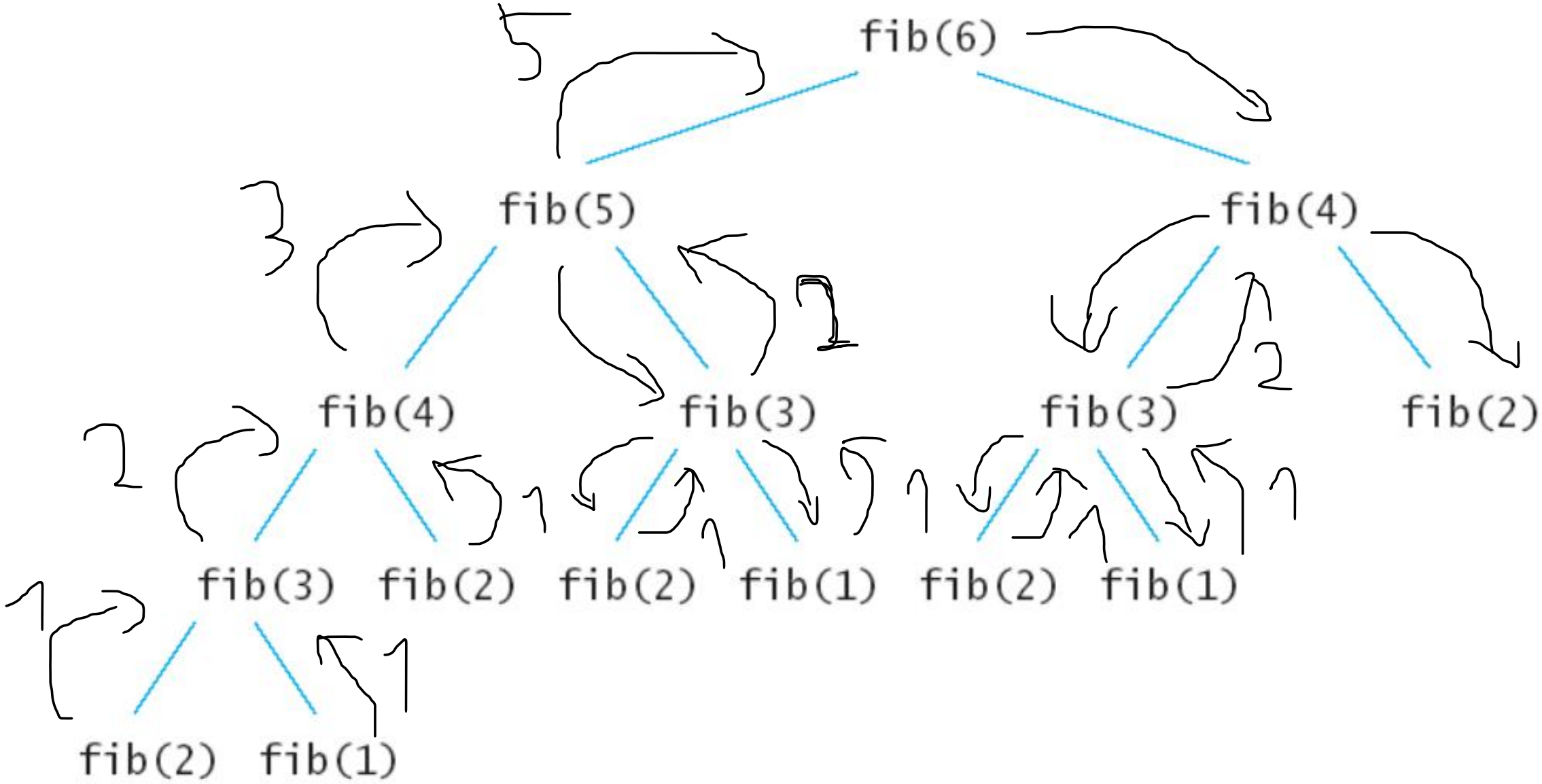
https://www.artofcse.com/learning/introduction-to-dynamic-programming



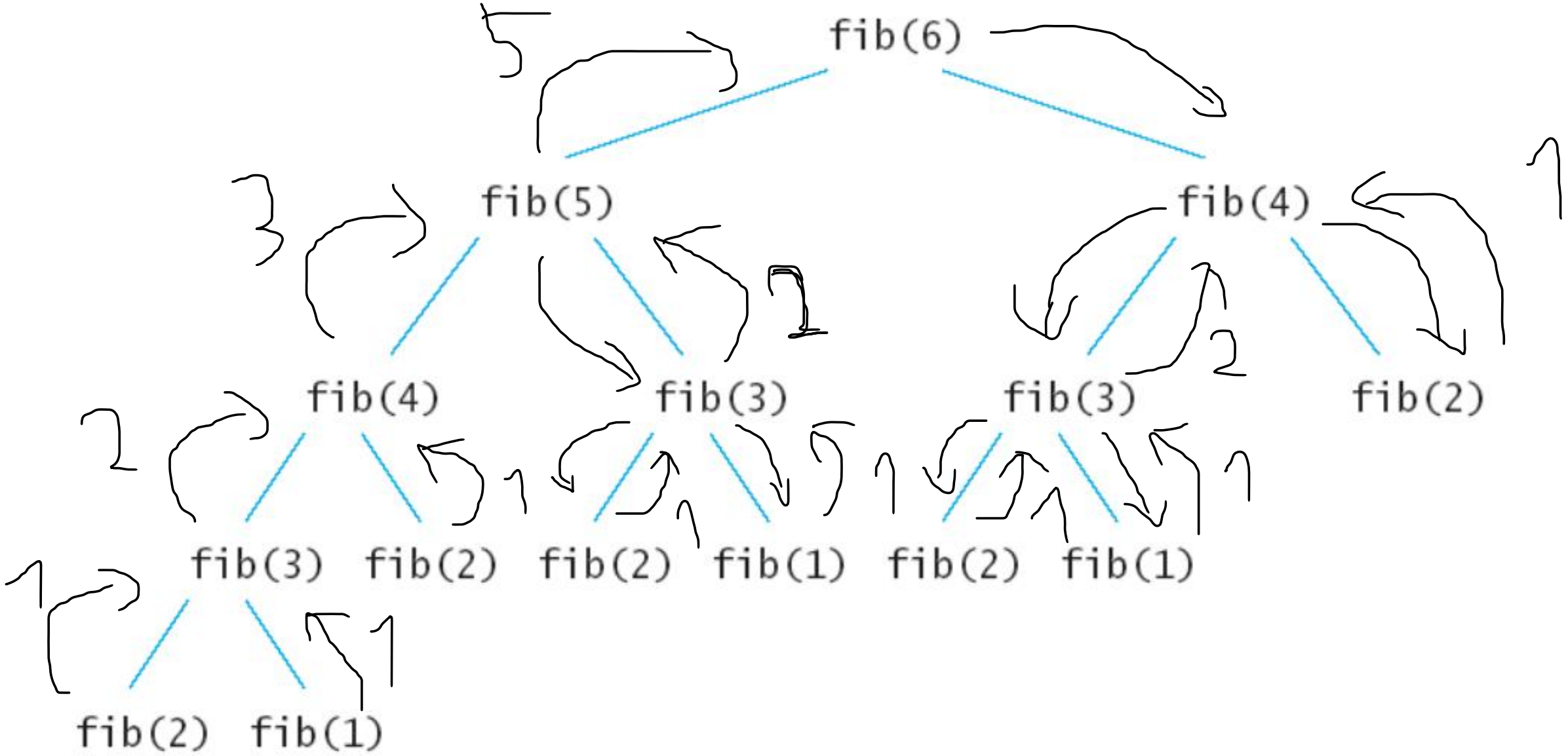
https://www.artofcse.com/learning/introduction-to-dynamic-programming



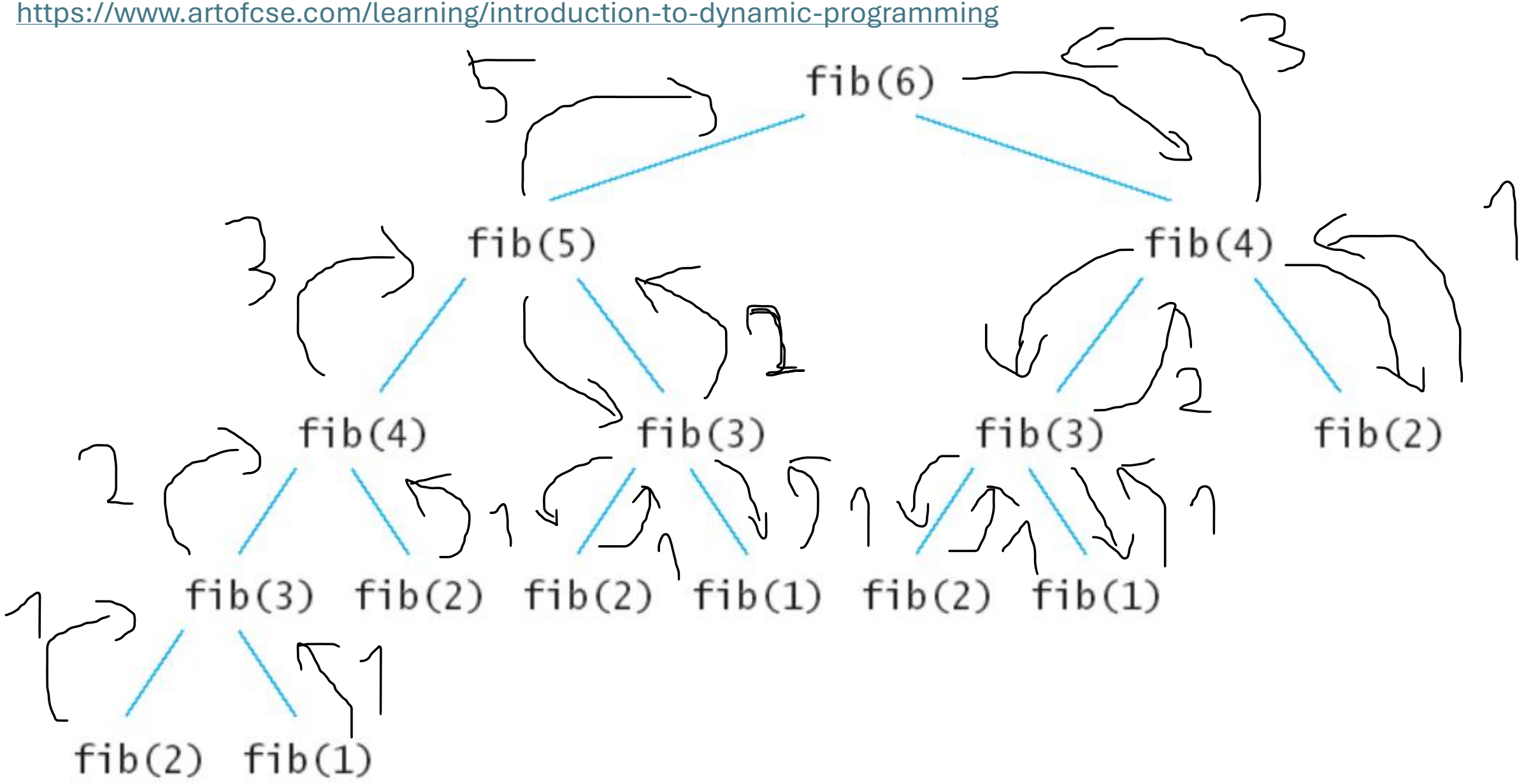
https://www.artofcse.com/learning/introduction-to-dynamic-programming



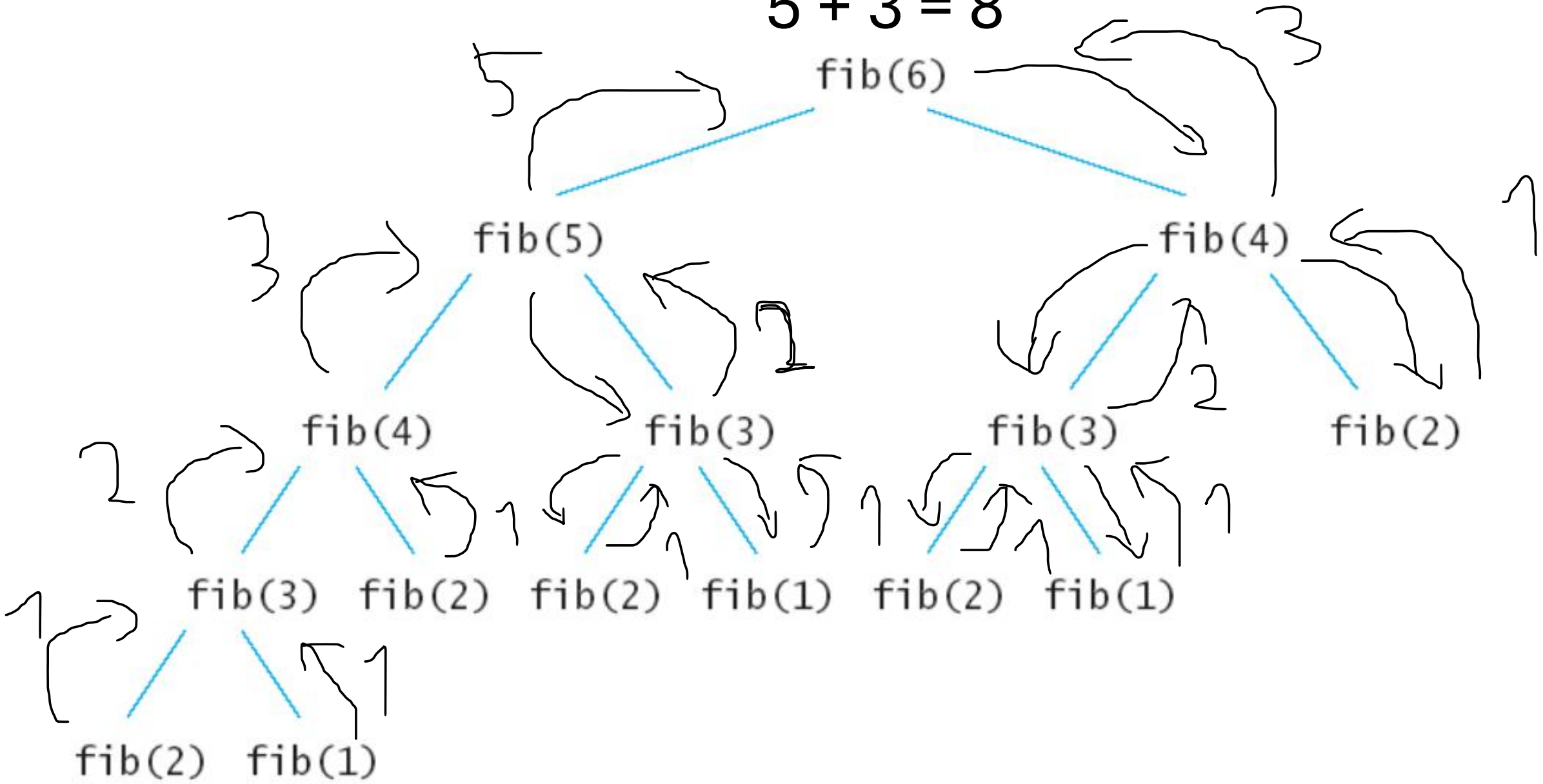
https://www.artofcse.com/learning/introduction-to-dynamic-programming



https://www.artofcse.com/learning/introduction-to-dynamic-programming



5 + 3 = 8


fib(6)
fib(5)
fib(4)
fib(4)
fib(3)
fib(3)
fib(2)
fib(3)
fib(2)
fib(2)
fib(1)
fib(2)
fib(1)
fib(2)
fib(1)

F.Y.I.




```
memo_size = 2**5
memo = [False] * memo_size
last_known_elem = 3
memo[1] = 1
memo[2] = 1
def fibo(n):
    if type(n) is not int or n < 1:
        raise ValueError("n must be an integer greater than 0")
    global memo, last_known_elem, memo_size
    if n < memo_size and memo[n]:
        return memo[n]
    for i in range(last_known_elem, n + 1):
        if i >= memo_size:
            memo += [False] * memo_size
            memo_size <<= 1
        memo[i] = memo[i - 1] + memo[i - 2]
    last_known_elem = n
    return memo[n]

```